



UNIVERSIDADE
ESTADUAL de LONDRINA

EWERTHON JOSÉ KUTZ

UTILIZAÇÃO DE SMALL LANGUAGE MODELS COM
CONHECIMENTO DE DOMÍNIO E LARGE LANGUAGE
MODELS EM SISTEMAS MULTIAGENTES PARA
INTERFACES TEXT-TO-SQL

LONDRINA

2026

EWERTHON JOSÉ KUTZ

**UTILIZAÇÃO DE SMALL LANGUAGE MODELS COM
CONHECIMENTO DE DOMÍNIO E LARGE LANGUAGE
MODELS EM SISTEMAS MULTIAGENTES PARA
INTERFACES TEXT-TO-SQL**

Dissertação apresentada ao Programa de
Mestrado em Ciência da Computação da
Universidade Estadual de Londrina para ob-
tenção do título de Mestre em Ciência da
Computação.

Orientador: Profa. Dra. Helen Cristina de
M. Senefonte

Coorientador: Prof(a). Dr(a). Nome do(a)
Coorientador(a)

LONDRINA

2026

Ficha de identificação da obra elaborada pelo autor, através do Programa de Geração Automática do Sistema de Bibliotecas da UEL

Sobrenome, Nome.

Título do Trabalho : Subtítulo do Trabalho / Nome Sobrenome. - Londrina, 2017.
100 f. : il.

Orientador: Nome do Orientador Sobrenome do Orientador.

Coorientador: Nome Coorientador Sobrenome Coorientador.

Dissertação (Mestrado em Ciência da Computação) - Universidade Estadual de Londrina, Centro de Ciências Exatas, Programa de Pós-Graduação em Ciência da Computação, 2017.

Inclui bibliografia.

1. Assunto 1 - Tese. 2. Assunto 2 - Tese. 3. Assunto 3 - Tese. 4. Assunto 4 - Tese. I. Sobrenome do Orientador, Nome do Orientador. II. Sobrenome Coorientador, Nome Coorientador. III. Universidade Estadual de Londrina. Centro de Ciências Exatas. Programa de Pós-Graduação em Ciência da Computação. IV. Título.

EWERTHON JOSÉ KUTZ

**UTILIZAÇÃO DE SMALL LANGUAGE MODELS COM
CONHECIMENTO DE DOMÍNIO E LARGE LANGUAGE
MODELS EM SISTEMAS MULTIAGENTES PARA
INTERFACES TEXT-TO-SQL**

Dissertação apresentada ao Programa de
Mestrado em Ciência da Computação da
Universidade Estadual de Londrina para ob-
tenção do título de Mestre em Ciência da
Computação.

BANCA EXAMINADORA

Orientador: Profa. Dra. Helen Cristina de
M. Senefonte
Universidade Estadual de Londrina

Prof. Dr. Segundo Membro da Banca
Universidade/Instituição do Segundo
Membro da Banca – Sigla instituição

Prof. Dr. Terceiro Membro da Banca
Universidade/Instituição do Terceiro
Membro da Banca – Sigla instituição

Prof. Ms. Quarto Membro da Banca
Universidade/Instituição do Quarto
Membro da Banca – Sigla instituição

Londrina, Data TBD de 2026.

*Este trabalho é dedicado às crianças adultas
que, quando pequenas, sonharam em se
tornar cientistas.*

AGRADECIMENTOS

Os agradecimentos principais são direcionados à Gerald Weber, Miguel Frasson, Leslie H. Watter, Bruno Parente Lima, Flávio de Vasconcellos Corrêa, Otavio Real Salvador, Renato Machnievscz¹ e todos aqueles que contribuíram para que a produção de trabalhos acadêmicos conforme as normas ABNT com L^AT_EX fosse possível.

Agradecimentos especiais são direcionados ao Centro de Pesquisa em Arquitetura da Informação² da Universidade de Brasília (CPAI), ao grupo de usuários *latex-br*³ e aos novos voluntários do grupo *abnT_EX2*⁴ que contribuíram e que ainda contribuirão para a evolução do abnT_EX2.

¹ Os nomes dos integrantes do primeiro projeto abnT_EX foram extraídos de <<http://codigolivre.org.br/projects/abntex/>>

² <<http://www.cpai.unb.br/>>

³ <<http://groups.google.com/group/latex-br>>

⁴ <<http://groups.google.com/group/abntex2>> e <<http://abntex2.googlecode.com/>>

*“Não vos amoldeis às estruturas deste mundo, mas transformai-vos pela renovação da mente, a fim de distinguir qual é a vontade de Deus: o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2))*

KUTZ, E. J.. **Utilização de Small Language Models com conhecimento de domínio e Large Language Models em sistemas multiagentes para interfaces Text-to-SQL**. 2026. 61f. Dissertação (Mestrado em Ciência da Computação) – Universidade Estadual de Londrina, Londrina, 2026.

RESUMO

Os recentes avanços em inteligência artificial têm impulsionado o uso de modelos de linguagem generativos de formas complexas e em aplicações cada vez mais avançadas, incluindo o uso de Large Language Models (LLMs) como agentes em sistemas inteligentes. Este trabalho propõe utilizar Small Language Models (SLMs) com conhecimento de domínio em comparação a LLMs em sistemas multiagentes colaborativos em interfaces Text-To-SQL. Para isso, esses modelos serão ajustados com métodos como qLoRA e few-shot prompting e avaliados como agentes em métricas de acurácia, corretude e custo-benefício por meio de datasets de interface de linguagem natural para SQL. Espera-se validar o uso de SLMs como uma solução eficaz frente a modelos maiores e mais custosos, além de desenvolver um framework de agentes colaborativos que pode ser reutilizado em outras aplicações.

Palavras-chave: Modelos de Linguagem. Sistemas Multiagentes. Text-to-SQL.

KUTZ, E. J.. Utilization of Small Language Models with domain knowledge and Large Language Models in multi-agent systems for Text-to-SQL interfaces.. 2026. 61p. Master's Thesis (Master in Science in Computer Science) – State University of Londrina, Londrina, 2026.

ABSTRACT

Recent advances in artificial intelligence have driven the use of generative language models in increasingly complex and advanced applications, including the deployment of Large Language Models (LLMs) as agents in intelligent systems. This work proposes the use of Small Language Models (SLMs) with domain knowledge, in comparison to LLMs, within collaborative multi-agent systems in Text-to-SQL interfaces. To achieve this, these models will be fine-tuned using methods such as qLoRA and few-shot prompting and evaluated as agents based on accuracy, correctness, and cost-effectiveness metrics using datasets designed for natural language to SQL interfaces. The expected outcome is to validate the use of SLMs as an effective solution compared to larger, more costly models, as well as to develop a collaborative agents framework that can be reused in other applications.

Keywords: Language Models. Multi-agent Systems. Text-to-SQL.

LISTA DE ILUSTRAÇÕES

Figura 1 – A delimitação do espaço	17
Figura 2 – Gráfico produzido em Excel e salvo como PDF. (Fonte: 1, p. 24)	18
Figura 3 – Grafico 1 da minipage. (Fonte: 1, p. 24)	18
Figura 4 – Grafico 2 da minipage. (Fonte: 1, p. 24)	18
Figura 5 – Espaço semântico e vetorial. Fonte: [2].	27
Figura 6 – Analogia do mecanismo de atenção com um dicionário difuso. Fonte: [2].	28
Figura 7 – Scaled Dot-Product Attention. Fonte: [3].	28
Figura 8 – Multi-Head Attention. Fonte: [4].	29
Figura 9 – Processo de ajuste fino. Fonte: [5].	33
Figura 10 – O Supervised Fine-Tuning (SFT) como extensão do treinamento do modelo base em conjuntos de dados especializados. Fonte: [2].	33
Figura 11 – RLHF: o aprendizado por reforço ajusta os parâmetros combinando pontuações de qualidade e similaridade para otimizar as saídas. Fonte: [2].	34
Figura 12 – Representação da técnica LoRA. Fonte: [6].	35
Figura 13 – Comparação de métodos: QLoRA aprimora o LoRA via quantização de 4 bits e otimizadores paginados. Fonte: [7].	36
Figura 14 – Arquitetura TALM (<i>Tool-Augmented Language Models</i>): ciclo de inte- ração entre modelo e ferramentas via interface de texto. Fonte: [8]. . . .	38
Figura 15 – Cenários de Interação para Sistemas Multiagentes com LLMs. Fonte: [9].	39
Figura 16 – Arquitetura multiagente no sistema OptiGuide: interação entre agentes de coordenação, escrita e salvaguarda. Fonte: [10].	40
Figura 17 – Fluxo de trabalho baseado em SOPs no MetaGPT. Fonte: [11].	41
Figura 18 – Framework de LLMs em Text-to-SQL. (Fonte: 12)	43
Figura 19 – Exemplo de falso negativo na métrica EM. As consultas buscam o peso máximo de formas distintas, levando a EM a classificar a geração como incorreta. (Fonte: 13)	43
Figura 20 – Exemplo de falso positivo na métrica EX. O filtro extra altera a semân- tica, mas retorna o mesmo resultado vazio, gerando uma classificação incorreta de acerto. (Fonte: 13)	44
Figura 21 – Arquitetura do DIN-SQL. (Fonte: 14)	44
Figura 22 – Arquitetura do CodeS. (Fonte: 15)	45
Figura 23 – Melhoria de desempenho (EX) obtida com LoRA. O ganho é menor em tarefas de alta complexidade. (Fonte: 16)	46
Figura 24 – Visão geral do framework MAC-SQL. (Fonte: 17)	46

LISTA DE TABELAS

Tabela 1 – Níveis de investigação	17
Tabela 2 – Tabela de conversão de acentuação.	25

LISTA DE ABREVIATURAS E SIGLAS

ABNT	Associação Brasileira de Normas Técnicas
BNDES	Banco Nacional de Desenvolvimento Econômico e Social
IBGE	Instituto Nacional de Geografia e Estatística
IBICT	Instituto Brasileiro de Informação em Ciência e Tecnologia
NBR	Norma Brasileira

SUMÁRIO

1	INTRODUÇÃO	14
2	RESULTADOS DE COMANDOS	16
	<i>Isto é uma sinopse de capítulo. A ABNT não traz nenhuma normatização a respeito desse tipo de resumo, que é mais comum em romances e livros técnicos.</i>	
2.1	Codificação dos arquivos: UTF8	16
2.2	Citações diretas	16
2.3	Notas de rodapé	17
2.4	Tabelas	17
2.5	Figuras	17
2.5.1	Figuras em <i>minipages</i>	18
2.6	Expressões matemáticas	19
2.7	Enumerações: alíneas e subalíneas	19
2.8	Espaçamento entre parágrafos e linhas	20
2.9	Inclusão de outros arquivos	21
2.10	Compilar o documento L^AT_EX	21
2.11	Remissões internas	22
2.12	Divisões do documento: seção	22
2.12.1	Divisões do documento: subseção	22
2.12.1.1	Divisões do documento: subsubseção	22
2.12.1.2	Divisões do documento: subsubseção	23
2.12.2	Divisões do documento: subseção	23
2.12.2.1	Divisões do documento: subsubseção	23
2.12.2.1.1	Isto é um parágrafo rotulado	23
2.12.2.1.2	Isto é outro parágrafo rotulado	23
2.13	Este é um exemplo de nome de seção longo. Ele deve estar alinhado à esquerda e a segunda e demais linhas devem iniciar logo abaixo da primeira palavra da primeira linha	23
2.14	Diferentes idiomas e hifenizações	23
2.15	Consulte o manual da classe abntex2	24
2.16	Referências bibliográficas	24
2.16.1	Acentuação de referências bibliográficas	25
2.17	Precisa de ajuda?	25
2.18	Você pode ajudar?	25

2.19	Quer customizar os modelos do abnT _E X2 para sua instituição ou universidade?	25
3	FUNDAMENTAÇÃO TEÓRICA	26
3.1	Modelos de Linguagem	26
3.2	Large Language Models e Small Language Models	29
3.3	Ajuste Fino (<i>Fine-tuning</i>)	32
3.4	Sistemas de Agentes de Inteligência Artificial	35
3.5	Interfaces Text-to-SQL	42
4	LOREM IPSUM DOLOR SIT AMET	47
4.1	Aliquam vestibulum fringilla lorem	47
4.1.1	Aliquam vestibulum fringilla lorem	47
4.1.1.1	Aliquam vestibulum fringilla lorem	47
5	LECTUS LOBORTIS CONDIMENTUM	48
5.1	Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae	48
6	NAM SED TELLUS SIT AMET LECTUS URNA ULLAMCORPER TRISTIQUE INTERDUM ELEMENTUM	49
6.1	Pellentesque sit amet pede ac sem eleifend consectetuer	49
7	CONCLUSÃO	50
	REFERÊNCIAS	51
	APÊNDICES	56
	APÊNDICE A – QUISQUE LIBERO JUSTO	57
	ANEXOS	58
	ANEXO A – MORBI ULTRICES RUTRUM LOREM.	59
	Trabalhos Publicados pelo Autor	60
	Índice	61

1 INTRODUÇÃO

Este documento e seu código-fonte são uma pequena adaptação do exemplo de uso fornecido pela equipe desenvolvedora da classe `abntex2`, atendendo a particularidades inseridas na classe `ABNT-DC-UEL.cls` que define o modelo para Trabalhos de Conclusão de Curso e Dissertações de Mestrado dos cursos do Departamento de Computação da Universidade Estadual de Londrina. Observe-se que o documento está preparado para impressão em frente e verso (opções `twoside` e `openright`) e que para gerar o índice remissivo deve-se utilizar o comando `makeindex`. Sugestões de melhorias ou problemas encontrados, favor notificar (Prof. Daniel S. Kaster – dkaster@uel.br).

Este documento e seu código-fonte são exemplos de referência de uso da classe `abntex2` e do pacote `abntex2cite`. O documento exemplifica a elaboração de trabalho acadêmico (tese, dissertação e outros do gênero) produzido conforme a ABNT NBR 14724:2011 *Informação e documentação - Trabalhos acadêmicos - Apresentação*.

A expressão “Modelo Canônico” é utilizada para indicar que `abnTEX2` não é modelo específico de nenhuma universidade ou instituição, mas que implementa tão somente os requisitos das normas da ABNT. Uma lista completa das normas observadas pelo `abnTEX2` é apresentada em 18.

Sinta-se convidado a participar do projeto `abnTEX2`! Acesse o site do projeto em <http://abntex2.googlecode.com/>. Também fique livre para conhecer, estudar, alterar e redistribuir o trabalho do `abnTEX2`, desde que os arquivos modificados tenham seus nomes alterados e que os créditos sejam dados aos autores originais, nos termos da “The L^AT_EX Project Public License”¹.

Encorajamos que sejam realizadas customizações específicas deste exemplo para universidades e outras instituições — como capas, folha de aprovação, etc. Porém, recomendamos que ao invés de se alterar diretamente os arquivos do `abnTEX2`, distribua-se arquivos com as respectivas customizações. Isso permite que futuras versões do `abnTEX2` não se tornem automaticamente incompatíveis com as customizações promovidas. Consulte 19 par mais informações.

Este documento deve ser utilizado como complemento dos manuais do `abnTEX2` [18, 20, 21] e da classe `memoir` [22].

Esperamos, sinceramente, que o `abnTEX2` aprimore a qualidade do trabalho que você produzirá, de modo que o principal esforço seja concentrado no principal: na contribuição científica.

¹ <http://www.latex-project.org/lppl.txt>

Equipe abnT_EX2

Lauro César Araujo

2 RESULTADOS DE COMANDOS

Isto é uma sinopse de capítulo. A ABNT não traz nenhuma normatização a respeito desse tipo de resumo, que é mais comum em romances e livros técnicos.

2.1 Codificação dos arquivos: UTF8

A codificação de todos os arquivos do `abnTEX22` é UTF8. É necessário que você utilize a mesma codificação nos documentos que escrever, inclusive nos arquivos de base bibliográficas `|.bib|`.

2.2 Citações diretas

Utilize o ambiente `citacao` para incluir citações diretas com mais de três linhas:

As citações diretas, no texto, com mais de três linhas, devem ser destacadas com recuo de 4 cm da margem esquerda, com letra menor que a do texto utilizado e sem aspas. No caso de documentos datilografados, deve-se observar apenas o recuo [23, 5.3].

Use o ambiente assim:

```
\begin{citacao}
As citações diretas, no texto, com mais de três linhas [...] deve-se observar
apenas o recuo \cite[5.3]{NBR10520:2002}.
\end{citacao}
```

O ambiente `citacao` pode receber como parâmetro opcional um nome de idioma previamente carregado nas opções da classe (Seção 2.14). Nesse caso, o texto da citação é automaticamente escrito em itálico e a hifenização é ajustada para o idioma selecionado na opção do ambiente. Por exemplo:

```
\begin{citacao}[english]
Text in English language in italic with correct hyphenation.
\end{citacao}
```

Tem como resultado:

Text in English language in italic with correct hyphenation.

Citações simples, com até três linhas, devem ser incluídas com aspas. Observe que em `LATEX` as aspas iniciais são diferentes das finais: “Amor é fogo que arde sem se ver”.

2.3 Notas de rodapé

As notas de rodapé são detalhadas pela NBR 14724:2011 na seção 5.2.1^{1,2,3}.

2.4 Tabelas

A Tabela 1 é um exemplo de tabela construída em $\text{\LaTeX}$.

Tabela 1 – Níveis de investigação. (Fonte: 25)

Nível de Inves- tigação	Insumos	Sistemas de Investigação	Produtos
Meta-nível	Filosofia da Ciência	Epistemologia	Paradigma
Nível do objeto	Paradigmas do metanível e evidências do nível inferior	Ciência	Teorias e modelos
Nível inferior	Modelos e métodos do nível do objeto e problemas do nível inferior	Prática	Solução de problemas

2.5 Figuras

Figuras podem ser criadas diretamente em $\text{\LaTeX}$, como o exemplo da Figura 1.

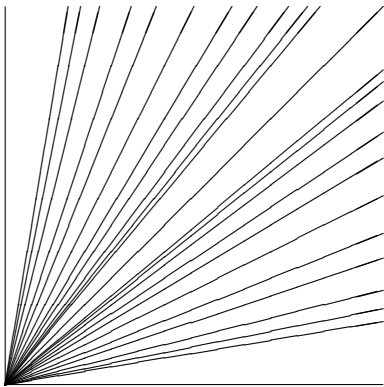


Figura 1 – A delimitação do espaço

Ou então figuras podem ser incorporadas de arquivos externos, como é o caso da Figura 2. Se a figura que ser incluída se tratar de um diagrama, um gráfico ou uma ilustração que você mesmo produza, priorize o uso de imagens vetoriais no formato PDF. Com isso, o tamanho do arquivo final do trabalho será menor, e as imagens terão uma apresentação

¹ As notas devem ser digitadas ou datilografadas dentro das margens, ficando separadas do texto por um espaço simples de entre as linhas e por filete de 5 cm, a partir da margem esquerda. Devem ser alinhadas, a partir da segunda linha da mesma nota, abaixo da primeira letra da primeira palavra, de forma a destacar o expoente, sem espaço entre elas e com fonte menor 24, 5.2.1.
² Caso uma série de notas sejam criadas sequencialmente, o $\text{\LaTeX}$ instrui o $\text{\LaTeX}$ para que uma vírgula seja colocada após cada número do expoente que indica a nota de rodapé no corpo do texto.
³ Verifique se os números do expoente possuem uma vírgula para dividi-los no corpo do texto.

melhor, principalmente quando impressas, uma vez que imagens vetoriais são perfeitamente escaláveis para qualquer dimensão. Nesse caso, se for utilizar o Microsoft Excel para produzir gráficos, ou o Microsoft Word para produzir ilustrações, exporte-os como PDF e os incorpore ao documento conforme o exemplo abaixo. No entanto, para manter a coerência no uso de software livre (já que você está usando $\text{\LaTeX}$ e $\text{\abnTeX}$), teste a ferramenta **InkScape** (<http://inkscape.org/>). Ela é uma excelente opção de código-livre para produzir ilustrações vetoriais, similar ao CorelDraw ou ao Adobe Illustrator. De todo modo, caso não seja possível utilizar arquivos de imagens como PDF, utilize qualquer outro formato, como JPEG, GIF, BMP, etc. Nesse caso, você pode tentar aprimorar as imagens incorporadas com o software livre **Gimp** (<http://www.gimp.org/>). Ele é uma alternativa livre ao Adobe Photoshop.

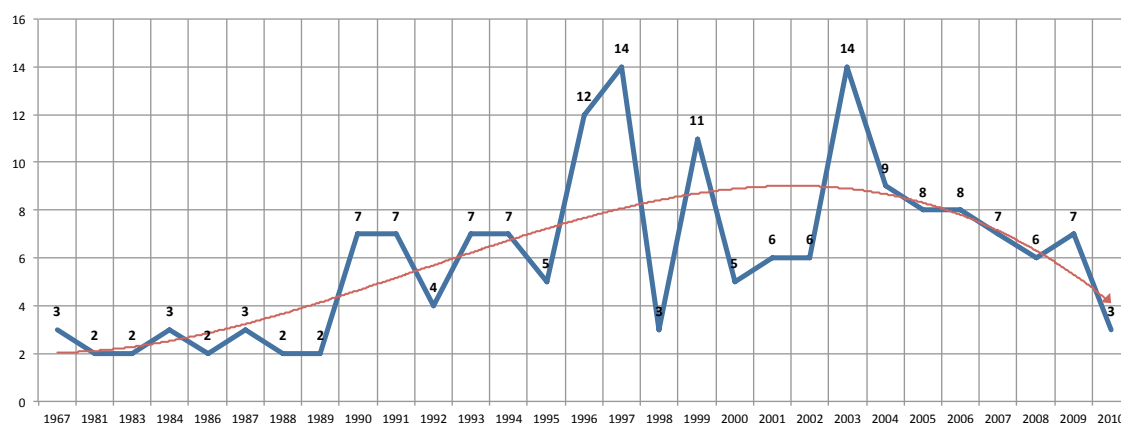


Figura 2 – Gráfico produzido em Excel e salvo como PDF. (Fonte: 1, p. 24)

2.5.1 Figuras em *minipages*

Minipages são usadas para inserir textos ou outros elementos em quadros com tamanhos e posições controladas. Veja o exemplo da Figura 3 e da Figura 4.

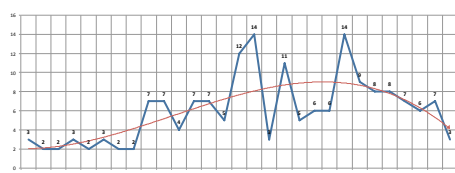


Figura 3 – Gráfico 1 da minipage. (Fonte: 1, p. 24)

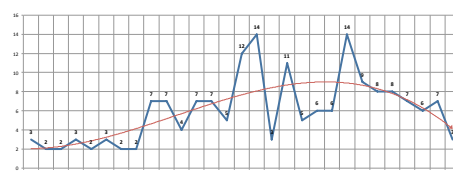


Figura 4 – Gráfico 2 da minipage. (Fonte: 1, p. 24)

Observe que, segundo a 24, seções 4.2.1.10 e 5.8, as ilustrações devem sempre ter numeração contínua e única em todo o documento:

Qualquer que seja o tipo de ilustração, sua identificação aparece na parte superior, precedida da palavra designativa (desenho, esquema, fluxograma, fotografia, gráfico, mapa, organograma, planta, quadro, retrato,

figura, imagem, entre outros), seguida de seu número de ordem de ocorrência no texto, em algarismos arábicos, travessão e do respectivo título. Após a ilustração, na parte inferior, indicar a fonte consultada (elemento obrigatório, mesmo que seja produção do próprio autor), legenda, notas e outras informações necessárias à sua compreensão (se houver). A ilustração deve ser citada no texto e inserida o mais próximo possível do trecho a que se refere. [24, seções 5.8]

2.6 Expressões matemáticas

Use o ambiente `equation` para escrever expressões matemáticas numeradas:

$$\forall x \in X, \quad \exists y \leq \epsilon \quad (2.1)$$

Escreva expressões matemáticas entre `$` e `$`, como em $\lim_{x \rightarrow \infty} \exp(-x) = 0$, para que fiquem na mesma linha.

Também é possível usar colchetes para indicar o início de uma expressão matemática que não é numerada.

$$\left| \sum_{i=1}^n a_i b_i \right| \leq \left(\sum_{i=1}^n a_i^2 \right)^{1/2} \left(\sum_{i=1}^n b_i^2 \right)^{1/2}$$

Consulte mais informações sobre expressões matemáticas em <http://code.google.com/p/abntex2/w/edit/Referencias>.

2.7 Enumerações: alíneas e subalíneas

Quando for necessário enumerar os diversos assuntos de uma seção que não possua título, esta deve ser subdividida em alíneas [26, 4.2]:

- a) os diversos assuntos que não possuam título próprio, dentro de uma mesma seção, devem ser subdivididos em alíneas;
- b) o texto que antecede as alíneas termina em dois pontos;
- c) as alíneas devem ser indicadas alfabeticamente, em letra minúscula, seguida de parêntese. Utilizam-se letras dobradas, quando esgotadas as letras do alfabeto;
- d) as letras indicativas das alíneas devem apresentar recuo em relação à margem esquerda;
- e) o texto da alínea deve começar por letra minúscula e terminar em ponto-e-vírgula, exceto a última alínea que termina em ponto final;
- f) o texto da alínea deve terminar em dois pontos, se houver subalínea;

- g) a segunda e as seguintes linhas do texto da alínea começa sob a primeira letra do texto da própria alínea;
- h) subalíneas [26, 4.3] devem ser conforme as alíneas a seguir:
 - as subalíneas devem começar por travessão seguido de espaço;
 - as subalíneas devem apresentar recuo em relação à alínea;
 - o texto da subalínea deve começar por letra minúscula e terminar em ponto-e-vírgula. A última subalínea deve terminar em ponto final, se não houver alínea subsequente;
 - a segunda e as seguintes linhas do texto da subalínea começam sob a primeira letra do texto da própria subalínea.
- i) no `abnTEX2` estão disponíveis os ambientes `incisos` e `subalineas`, que em suma são o mesmo que se criar outro nível de `alineas`, como nos exemplos à seguir:
 - *Um novo inciso em itálico;*
- j) Alínea em **negrito**:
 - *Uma subalínea em itálico;*
 - *Uma subalínea em itálico e sublinhado;*
- k) Última alínea com *ênfase*.

2.8 Espaçamento entre parágrafos e linhas

O tamanho do parágrafo, espaço entre a margem e o início da frase do parágrafo, é definido por:

```
\setlength{\parindent}{1.3cm}
```

Por padrão, não há espaçamento no primeiro parágrafo de cada início de divisão do documento (Seção 2.12). Porém, você pode definir que o primeiro parágrafo também seja indentado, como é o caso deste documento. Para isso, apenas inclua o pacote `indentfirst` no preâmbulo do documento:

```
\usepackage{indentfirst}      % Indenta o primeiro parágrafo de cada seção.
```

O espaçamento entre um parágrafo e outro pode ser controlado por meio do comando:

```
\setlength{\parskip}{0.2cm}  % tente também \onelineskip
```

O controle do espaçamento entre linhas é definido por:

```
\OnehalfSpacing      % espaçamento um e meio (padrão);
\DoubleSpacing       % espaçamento duplo
\SingleSpacing       % espaçamento simples
```

Para isso, também estão disponíveis os ambientes:

```
\begin{SingleSpace} ... \end{SingleSpace}
\begin{Spacing}{hfactori} ... \end{Spacing}
\begin{OnehalfSpace} ... \end{OnehalfSpace}
\begin{OnehalfSpace*} ... \end{OnehalfSpace*}
\begin{DoubleSpace} ... \end{DoubleSpace}
\begin{DoubleSpace*} ... \end{DoubleSpace*}
```

Para mais informações, consulte 22, p. 47-52 e 135.

2.9 Inclusão de outros arquivos

É uma boa prática dividir o seu documento em diversos arquivos, e não apenas escrever tudo em um único. Esse recurso foi utilizado neste documento. Para incluir diferentes arquivos em um arquivo principal, de modo que cada arquivo incluído fique em uma página diferente, utilize o comando:

```
\include{documento-a-ser-incluido}      % sem a extensão .tex
```

Para incluir documentos sem quebra de páginas, utilize:

```
\input{documento-a-ser-incluido}      % sem a extensão .tex
```

2.10 Compilar o documento L^AT_EX

Geralmente os editores L^AT_EX, como o TeXlipse⁴, o Texmaker⁵, entre outros, compilam os documentos automaticamente, de modo que você não precisa se preocupar com isso.

No entanto, você pode compilar os documentos L^AT_EX usando os seguintes comandos, que devem ser digitados no *Prompt de Comandos* do Windows ou no *Terminal* do Mac ou do Linux:

⁴ <<http://texlipse.sourceforge.net/>>

⁵ <<http://www.xmlmath.net/texmaker/>>

```

pdflatex ARQUIVO_PRINCIPAL.tex
bibtex ARQUIVO_PRINCIPAL.aux
makeindex ARQUIVO_PRINCIPAL.idx
makeindex ARQUIVO_PRINCIPAL.nlo -s nomencl.ist -o ARQUIVO_PRINCIPAL.nls
pdflatex ARQUIVO_PRINCIPAL.tex
pdflatex ARQUIVO_PRINCIPAL.tex

```

2.11 Remissões internas

Ao nomear a Tabela 1 e a Figura 1, apresentamos um exemplo de remissão interna, que também pode ser feita quando indicamos o Capítulo 2, que tem o nome *RESULTADOS DE COMANDOS*. O número do capítulo indicado é 2, que se inicia à Página 16⁶. Veja a Seção 2.12 para outros exemplos de remissões internas entre seções, subseções e subsubseções.

O código usado para produzir o texto desta seção é:

Ao nomear a `\autoref{tab-nivinv}` e a `\autoref{fig_circulo}`, apresentamos um exemplo de remissão interna, que também pode ser feita quando indicamos o `\autoref{cap_exemplos}`, que tem o nome `\emph{\nameref{cap_exemplos}}`. O número do capítulo indicado é `\ref{cap_exemplos}`, que se inicia à `\autopageref{cap_exemplos}`. O número da página de uma remissão pode ser obtida também assim:

`\pageref{cap_exemplos}`.

Veja a `\autoref{sec-divisoes}` para outros exemplos de remissões internas entre seções, subseções e subsubseções.

2.12 Divisões do documento: seção

Esta seção testa o uso de divisões de documentos. Esta é a Seção 2.12. Veja a Subseção 2.12.1.

2.12.1 Divisões do documento: subseção

Isto é uma subseção. Veja a Subsubseção 2.12.1.1, que é uma `subsubsection` do L^AT_EX, mas é impressa chamada de “subseção” porque no Português não temos a palavra “subsubseção”.

2.12.1.1 Divisões do documento: subsubseção

Isto é uma subsubseção.

⁶ O número da página de uma remissão pode ser obtida também assim: 16.

2.12.1.2 Divisões do documento: subsubseção

Isto é outra subsubseção.

2.12.2 Divisões do documento: subseção

Isto é uma subseção.

2.12.2.1 Divisões do documento: subsubseção

Isto é mais uma subsubseção da Subseção 2.12.2.

2.12.2.1.1 Isto é um parágrafo rotulado

Este é um parágrafo na parágrafo 2.12.2.1.1.

2.12.2.1.2 Isto é outro parágrafo rotulado

Este é outro parágrafo na parágrafo 2.12.2.1.2.

2.13 Este é um exemplo de nome de seção longo. Ele deve estar alinhado à esquerda e a segunda e demais linhas devem iniciar logo abaixo da primeira palavra da primeira linha

Isso atende à norma 24, seções de 5.2.2 a 5.2.4 e 26, seções de 3.1 a 3.8.

2.14 Diferentes idiomas e hifenizações

Para usar hifenizações de diferentes idiomas, inclua nas opções do documento o nome dos idiomas que o seu texto contém. Por exemplo:

```
\documentclass[12pt,openright,twoside,a4paper,english,french,spanish,brazil]{abntex}
```

O idioma português-brasileiro (`brazil`) é incluído automaticamente pela classe `abntex2`. Porém, mesmo assim a opção `brazil` deve ser informada como a última opção da classe para que todos os pacotes reconheçam o idioma. Vale ressaltar que a última opção de idioma é a utilizada por padrão no documento. Desse modo, caso deseje escrever um texto em inglês que tenha citações em português e em francês, você deveria usar o preâmbulo como abaixo:

```
\documentclass[12pt,openright,twoside,a4paper,french,brazil,english]{abntex2}
```


A lista completa de idiomas suportados, bem como outras opções de hifenização, estão disponíveis em 27, p. 5-6.

Exemplo de hifenização em inglês⁷:

Text in English language. This environment switches all language-related definitions, like the language specific names for figures, tables etc. to the other language. The starred version of this environment typesets the main text according to the rules of the other language, but keeps the language specific string for ancillary things like figures, in the main language of the document. The environment hyphenrules switches only the hyphenation patterns used; it can also be used to disallow hyphenation by using the language name ‘nohyphenation’.

O idioma geral do texto por ser alterado como no exemplo seguinte:

```
\selectlanguage{english}
```

Isso altera automaticamente a hifenização e todos os nomes constantes de referências do documento para o idioma inglês. Consulte o manual da classe [18] para obter orientações adicionais sobre internacionalização de documentos produzidos com abnTeX2.

A Seção 2.2 descreve o ambiente `citacao` que pode receber como parâmetro um idioma a ser usado na citação.

2.15 Consulte o manual da classe `abntex2`

Consulte o manual da classe `abntex2` [18] para uma referência completa das macros e ambientes disponíveis.

Além disso, o manual possui informações adicionais sobre as normas ABNT observadas pelo abnTeX2 e considerações sobre eventuais requisitos específicos não atendidos, como o caso da 24, seção 5.2.2, que especifica o espaçamento entre os capítulos e o início do texto, regra propositalmente não atendida pelo presente modelo.

2.16 Referências bibliográficas

A formatação das referências bibliográficas conforme as regras da ABNT são um dos principais objetivos do abnTeX2. Consulte os manuais 20 e 21 para obter informações sobre como utilizar as referências bibliográficas.

⁷ Extraído de: <<http://en.wikibooks.org/wiki/LaTeX/Internationalization>>

2.16.1 Acentuação de referências bibliográficas

Normalmente não há problemas em usar caracteres acentuados em arquivos bibliográficos (*.bib). Porém, como as regras da ABNT fazem uso quase abusivo da conversão para letras maiúsculas, é preciso observar o modo como se escreve os nomes dos autores. Na Tabela 2 você encontra alguns exemplos das conversões mais importantes. Preste atenção especial para ‘ç’ e ‘í’ que devem estar envoltos em chaves. A regra geral é sempre usar a acentuação neste modo quando houver conversão para letras maiúsculas.

Tabela 2 – Tabela de conversão de acentuação.

acento	bibtex
à á ã	\‘a \’a \~a
í	{\’\i}
ç	{\c c}

2.17 Precisa de ajuda?

Consulte a FAQ com perguntas frequentes e comuns no portal do abnT_EX2: <<https://code.google.com/p/abntex2/wiki/FAQ>>.

Inscreva-se no grupo de usuários L^AT_EX: <<http://groups.google.com/group/latex-br>>, tire suas dúvidas e ajude outros usuários.

Participe também do grupo de desenvolvedores do abnT_EX2: <<http://groups.google.com/group/abntex2>> e faça sua contribuição à ferramenta.

2.18 Você pode ajudar?

Sua contribuição é muito importante! Você pode ajudar na divulgação, no desenvolvimento e de várias outras formas. Veja como contribuir com o abnT_EX2 em <<https://code.google.com/p/abntex2/wiki/ComoContribuir>>.

2.19 Quer customizar os modelos do abnT_EX2 para sua instituição ou universidade?

Veja como customizar o abnT_EX2 em: <<https://code.google.com/p/abntex2/wiki/ComoCustomizar>>.

3 FUNDAMENTAÇÃO TEÓRICA

3.1 Modelos de Linguagem

Um modelo de linguagem computacional consiste em um sistema probabilístico capaz de prever a próxima unidade de texto em uma sequência, condicionada às anteriores [4]. Embora frequentemente referidas como "palavras", as unidades fundamentais processadas pelos modelos modernos denominam-se *tokens*. Estes podem representar palavras inteiras, subpalavras ou caracteres, tornando a *tokenização* a etapa primária de engenharia de características [2, 4]. Para o processamento, tais *tokens* são convertidos em vetores numéricos multidimensionais. Dada a complexidade linguística, uma representação simples desse espaço resulta em vetores de altíssima dimensionalidade e esparsidade e torna seu processamento uma tarefa extremamente complexa, fenômeno conhecido como a "maldição da dimensionalidade"[2].

Historicamente, abordagens iniciais como o *Bag-of-Words* (BoW) tratavam o texto apenas como uma contagem de frequência, ignorando a ordem sequencial e a estrutura sintática [28]. A superação dessa limitação ocorreu com o desenvolvimento de representações distribuídas, fundamentadas na hipótese distribucional de linguistas como Harris e Firth (década de 1950), que postula que palavras em contextos similares compartilham significados semelhantes [29]. Contudo, essas representações ganharam viabilidade prática apenas recentemente com técnicas eficientes como o Word2Vec, que consolidaram o uso de *embeddings* [4, 30].

Os Modelos de Linguagem Neurais (NLMs) operam sobre esse espaço vetorial denso, onde a distância euclidiana entre os vetores reflete a similaridade semântica entre os termos. Diferentemente dos modelos de contagem, os NLMs generalizam o aprendizado: ao identificar que dois *tokens* compartilham contextos vetoriais próximos, o modelo transfere o conhecimento de um para o outro [30]. Os *embeddings*, portanto, solucionam o problema da dimensionalidade ao projetar o vocabulário em um espaço contínuo de baixa dimensão, capturando relações semânticas complexas [28], conforme ilustrado na Figura 5.

A natureza algébrica dos *embeddings* permite operações vetoriais que refletem analogias linguísticas — como a clássica operação "rei - homem + mulher = rainha" — e a identificação de sinonímia que modelos baseados em contagem tratariam como entidades disjuntas [4, 28].

Em 2017, a introdução da arquitetura *Transformers* estabeleceu um novo paradigma no processamento de linguagem natural [28]. Ao contrário das Redes Neurais Recorrentes (RNNs) e LSTMs, que processam dados sequencialmente e degradam em depen-

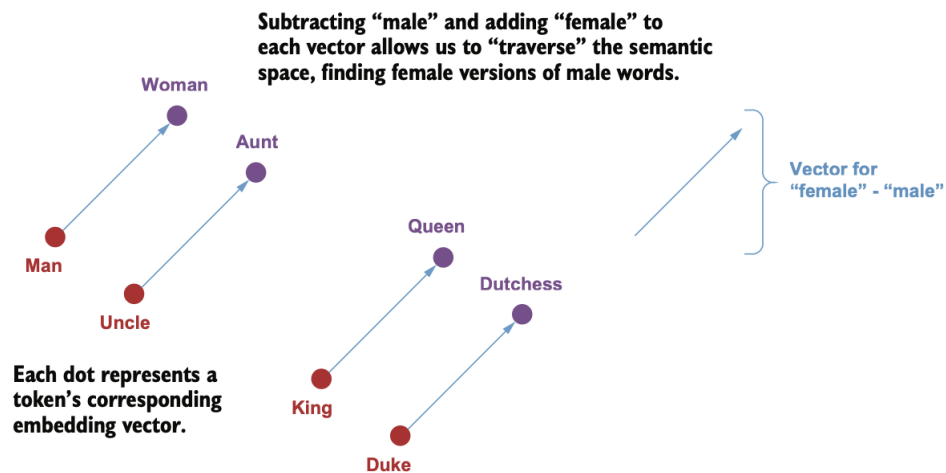


Figura 5 – Espaço semântico e vetorial. Fonte: [2].

dências de longa distância, os *Transformers* utilizam mecanismos de atenção para modelar dependências globais entre a entrada e a saída de forma paralela [28].

Embora a arquitetura original do *Transformer* [3] integrasse blocos de codificação e decodificação, a evolução da área conduziu à especialização dessas estruturas em três categorias arquiteturais [2]:

- **Modelos *Encoder-only*** (e.g., BERT): otimizados para tarefas de compreensão e classificação;
- **Modelos *Decoder-only*** (e.g., família GPT): focados na geração autorregressiva de texto;
- **Modelos *Encoder-Decoder*** (e.g., T5): mantêm a estrutura híbrida para tradução e sumarização.

O fluxo de processamento nos *Transformers* inicia-se na conversão dos *tokens* em representações vetoriais contínuas. Uma característica intrínseca dessa arquitetura é o processamento paralelo, que confere eficiência computacional, mas remove a informação implícita de ordem sequencial [4]. Para reintroduzir a noção de sequência, utilizam-se vetores de codificação posicional (*positional encodings*). Estes vetores, baseados em funções senoidais ou parâmetros aprendidos, são somados aos *embeddings* de entrada, permitindo que o modelo distinga a posição relativa dos termos (e.g., diferenciando sujeito de objeto) [29].

O mecanismo de atenção (*attention mechanism*) constitui o núcleo da capacidade do *Transformer* em contextualizar relações distantes. Para cada *token*, o modelo aprende três projeções lineares: *Query* (Q), *Key* (K) e *Value* (V) [28]. Estabelece-se uma analogia desse processo com um sistema de recuperação em um "dicionário difuso" (Figura 6) [2].

Diferente de uma busca exata, a *Query* de um *token* é comparada probabilisticamente com as *Keys* de todo o contexto para ponderar a recuperação dos valores.

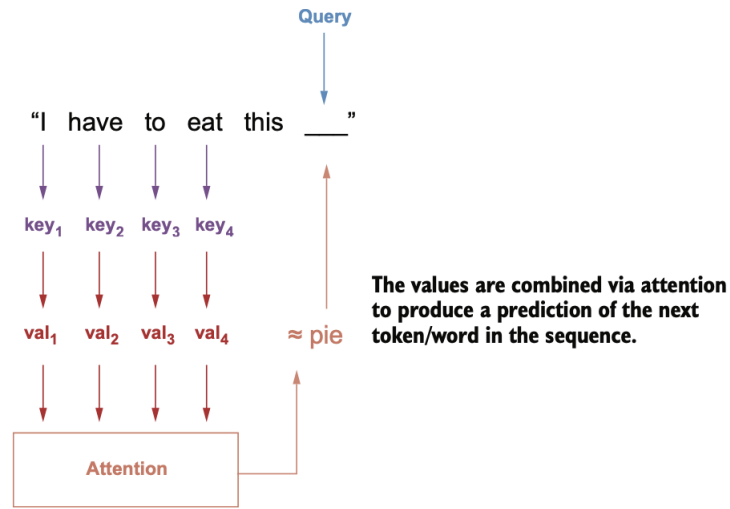


Figura 6 – Analogia do mecanismo de atenção com um dicionário difuso. Fonte: [2].

Matematicamente, essa comparação ocorre via produto escalar entre Q e K. Para garantir estabilidade numérica de gradientes, os resultados são divididos pela raiz quadrada da dimensão dos vetores ($\sqrt{d_k}$), técnica denominada *Scaled Dot-Product Attention* [3]. Aplica-se então uma função *softmax* para normalizar os escores em uma distribuição de probabilidade. Esses pesos resultantes ponderam a soma dos vetores *Value* (V), gerando uma representação final que captura o contexto semântico completo (Figura 7) [3].

Scaled Dot-Product Attention

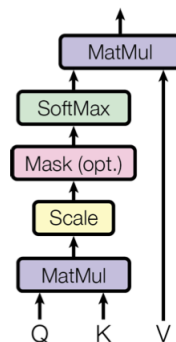


Figura 7 – Scaled Dot-Product Attention. Fonte: [3].

Para capturar simultaneamente diferentes nuances linguísticas (sintaxe, semântica, correferências), adota-se o *Multi-Head Attention*. Múltiplos mecanismos ("cabeças") operam em paralelo em subespaços representacionais distintos. A concatenação e projeção

linear dessas saídas proporcionam uma compreensão robusta, onde cada cabeça foca em aspectos complementares da sentença (Figura 8) [28, 29].

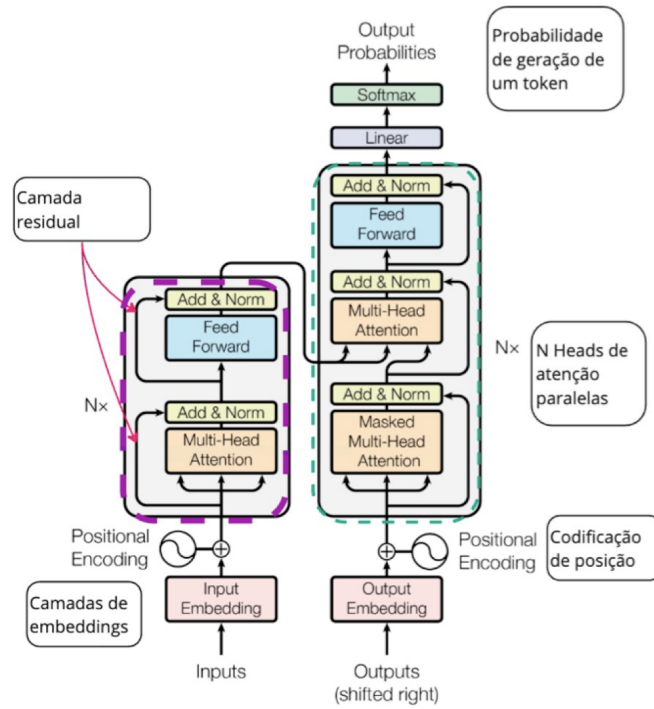


Figura 8 – Multi-Head Attention. Fonte: [4].

Nos modelos generativos (*decoder-only*), o processo é autorregressivo. A saída das camadas do *Transformer* atravessa uma etapa de *unembedding* seguida de uma função *softmax*, convertendo os vetores latentes em uma distribuição de probabilidade sobre o vocabulário [2, 29]. A seleção final do *token* depende de estratégias de amostragem (*sampling*). Parâmetros como a temperatura introduzem estocasticidade, permitindo criatividade ou alucinação. A natureza não-determinística é ilustrada com o exemplo de um modelo completando a frase "I love to eat" com "chalk"(giz); caso a amostragem selecione essa opção de baixa probabilidade, o modelo tentará justificar a escolha no contexto subsequente (e.g., citando uma condição médica), demonstrando a coerência interna independente da veracidade factual [2].

3.2 Large Language Models e Small Language Models

Os modelos de linguagem baseados em *Transformers* representam uma nova era no Processamento de Linguagem Natural (PLN) [4]. O primeiro sistema a ganhar notoriedade global, devido ao desempenho inédito em tarefas generalistas — quebrando recordes em 11 delas —, foi o BERT [31]. Posteriormente, o ChatGPT 3.5 elevou essas arquiteturas ao protagonismo da Inteligência Artificial, consolidando a capacidade de diálogo por meio do ajuste fino supervisionado e do *Reinforcement Learning from Human Feedback* (RLHF)

[5]. Embora sem definição formal rígida, o sucesso desse sistema popularizou o conceito de *Large Language Models* (LLMs), associando sua competência à escala massiva de centenas de bilhões de parâmetros.

Essa associação fundamenta-se em estudos preliminares que indicavam que o desempenho dos modelos de linguagem melhora continuamente conforme o aumento do tamanho da rede, do volume de dados e do poder computacional, obedecendo a leis de escala previsíveis [32]. Tais descobertas forneceram a justificativa científica para a corrida por modelos com centenas de bilhões de parâmetros, estabelecendo o dogma de que “maior é melhor” que dominou o desenvolvimento de LLMs nos anos subsequentes.

Adicionalmente, observa-se nesses sistemas o fenômeno das “habilidades emergentes”. Trata-se de capacidades que não podem ser previstas pela extrapolação linear do desempenho de modelos menores (como o BERT): a performance permanece próxima ao aleatório até que um limiar crítico de escala seja atingido, momento em que ocorre um salto qualitativo substancial [33]. Documentam-se múltiplos exemplos dessas emergências, incluindo raciocínio aritmético, compreensão multitarefa (MMLU) e seguimento de instruções complexas, que se manifestam tipicamente em faixas entre 10 bilhões e 500 bilhões de parâmetros, variando conforme a arquitetura [33, 34].

Contudo, a premissa de que o aumento de escala é o único vetor de qualidade foi contestada. Evidências apontaram que muitos LLMs apresentavam sinais de “subtreinamento” como consequência do foco excessivo no tamanho do modelo em detrimento do volume e qualidade de dados. Argumenta-se que, para um treinamento computacionalmente ótimo, o tamanho do modelo e o número de *tokens* de treinamento devem ser escalados proporcionalmente [35]. A descoberta de que modelos menores, quando treinados com maior volume de dados, podem superar arquiteturas significativamente maiores desafiou as leis de escala tradicionais. O modelo Chinchilla (70B), por exemplo, superou uniformemente modelos como Gopher (280B) e GPT-3 (175B) em diversas tarefas de avaliação, utilizando quatro vezes mais dados de treino [35]. Esses resultados estabeleceram as bases para o desenvolvimento de *Small Language Models* (SLMs) especializados, sugerindo que a eficiência do treinamento e a qualidade dos dados podem compensar a redução de parâmetros.

Pesquisas subsequentes corroboraram que a “inteligência” do modelo não é apenas subproduto do volume de parâmetros. A família Llama, com arquiteturas de até 13 bilhões de parâmetros focadas em eficiência de inferência e exposição a dados de alta qualidade, superou o GPT-3 (175B) na maioria dos *benchmarks*, apesar de ser treze vezes menor em número de parâmetros [36].

Paralelamente, a fundamentação da família Phi demonstrou que a “qualidade didática” dos dados permite que SLMs — definidos como modelos com menos de 10 bilhões de parâmetros — superem LLMs em raciocínio lógico e codificação [37]. O modelo phi-

1 (1,3B), por exemplo, obteve desempenho superior ao StarCoder (15,5B) no *benchmark* HumanEval. Para atingir tais resultados, o ajuste fino (*fine-tuning*) mostra-se crucial para desbloquear capacidades e equiparar o desempenho aos modelos massivos, tendência também observada na família Flan-T5, cuja variante de 3 bilhões de parâmetros superou o GPT-3 no *benchmark* MMLU em 9% [38].

Embora os LLMs representem o estado da arte em performance bruta, os SLMs consolidam-se como alternativas promissoras devido à menor latência e à viabilidade de execução local. Essa descentralização mitiga riscos de privacidade inerentes ao processamento em nuvem, fator crítico dado que os LLMs mais potentes são proprietários e fechados [39]. Quando ajustados e aplicados a domínios específicos - como o de saúde -, SLMs com menos de 1,6 bilhão de parâmetros alcançaram 75,4% de precisão no Pub-MedQA, superando os 74,4% do GPT-4 em cenários *few-shot* [39].

Além da eficácia, os SLMs endereçam o dilema energético. Modelos menores podem atingir desempenho competitivo gerando emissões de CO₂ entre 660 e 13.000 vezes inferiores às de um LLM massivo [40]. Apesar de ainda apresentarem limitações em raciocínio complexo e generalista, a tendência atual aponta para sistemas híbridos que mesclam ambos os tipos de modelos para otimizar a relação custo-benefício [34].

A transição para modelos compactos é particularmente estratégica para sistemas de agentes de IA. A predominância atual de LLMs em agentes mostra-se muitas vezes excessiva e desalinhada com demandas de escopo restrito. Os SLMs fornecem uma alternativa viável para o futuro dessa área, baseando-se em três pilares: capacidade técnica suficiente para sub-tarefas, adequação operacional e eficiência econômica [39, 41]:

- **Capacidade Técnica:** SLMs modernos demonstram aptidão para tarefas críticas de agentes, como uso de ferramentas (*tool calling*) e seguimento de instruções. O modelo Hymba-1.5B, por exemplo, supera modelos de 13B em precisão de instrução, enquanto a série Phi-2 (2,7B) atinge métricas de geração de código equivalentes a modelos de 30B, com velocidade 15 vezes superior [41].
- **Eficiência e Economia:** A inferência de um SLM de 7B é de 10 a 30 vezes mais econômica em latência e energia do que a de um LLM de 175B. Tal eficiência é vital, considerando que a inferência pode representar até 90% do consumo energético do ciclo de vida do modelo [41].
- **Agilidade e Especialização:** O ajuste fino de SLMs pode ser concluído em poucas horas, permitindo a rápida integração ou correção de comportamentos, em contraste com as semanas exigidas por modelos massivos. Isso favorece arquiteturas modulares onde SLMs atuam como especialistas interagindo com LLMs centrais [41].

Apesar dos avanços, modelos baseados em *Transformers* — independentemente da escala — enfrentam desafios estruturais. O custo computacional do mecanismo de atenção cresce quadraticamente com o comprimento da sequência, dificultando o processamento de contextos longos [29]. Para SLMs, essa restrição na "janela de contexto" pode inviabilizar a execução de tarefas complexas que exigem grande recuperação de informação [39].

Adicionalmente, o treinamento de modelos massivos demanda recursos exorbitantes. Estima-se que o treinamento do GPT-3 (175B) tenha emitido 550 toneladas métricas de CO₂, e com as leis de escala ainda vigentes, a demanda por recursos tende a crescer, exacerbando o impacto ambiental [40, 34]. Questões de confiabilidade também permanecem críticas: as alucinações — geração de informações plausíveis, porém incorretas — persistem tanto em LLMs quanto em SLMs. Esse fenômeno, decorrente da natureza probabilística de previsão de *tokens* e de dados ruidosos, revela uma lacuna na capacidade de monitoramento interno e verificação factual dessas ferramentas [34, 39]. Por fim, a interpretabilidade permanece um desafio aberto, visto que a complexidade das redes neurais profundas dificulta a compreensão mecanística de como o modelo deriva suas respostas, mantendo o estigma de "caixas pretas" [29].

3.3 Ajuste Fino (*Fine-tuning*)

O ajuste fino constitui o método essencial para alinhar modelos de linguagem pré-treinados à intenção humana, transformando sua capacidade bruta de previsão de texto na habilidade de seguir instruções de forma útil e segura [5]. A metodologia estrutura-se em três etapas distintas: primeiramente, o *Supervised Fine-Tuning* (SFT), onde o modelo é ajustado com um conjunto de dados de demonstrações humanas do comportamento desejado; em seguida, o treinamento de um Modelo de Recompensa (*Reward Model*), criado a partir de comparações onde humanos classificam diferentes saídas; e, por fim, a otimização via *Reinforcement Learning* utilizando o algoritmo PPO, que refina a política para maximizar a pontuação de recompensa [5]. Esse processo atua como a ponte que corrige o desalinhamento entre o objetivo de pré-treinamento e a necessidade de agir conforme as instruções do usuário, viabilizando as aplicações atuais de LLMs [42].

Operacionalmente, o SFT segue a lógica fundamental do pré-treinamento, utilizando a predição do próximo *token* para internalizar informações de novos documentos [5]. A distinção reside no ponto de partida e na especificidade dos dados: enquanto o modelo base inicia com parâmetros aleatórios expostos a volumes massivos de dados genéricos, o SFT reutiliza os pesos pré-treinados, ajustando-os via gradiente descendente em conjuntos altamente curados. Essa reutilização permite que o modelo retenha o conhecimento linguístico geral enquanto assimila nuances e formatos especializados [2].

Entretanto, o ajuste fino impõe desafios intrínsecos, notadamente o esquecimento

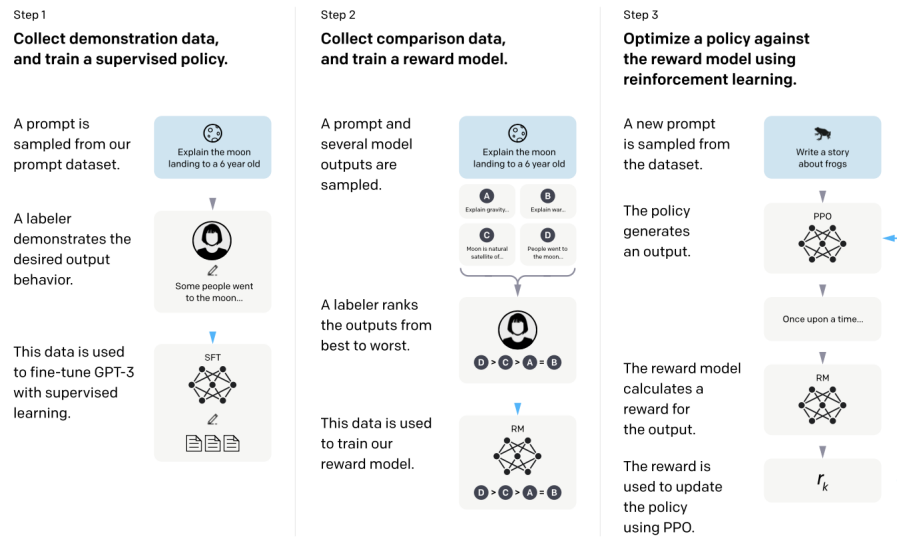


Figura 9 – Processo de ajuste fino. Fonte: [5].

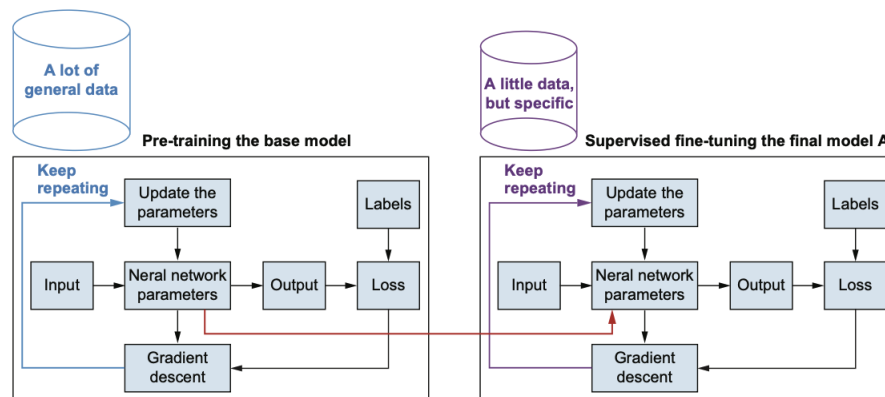


Figura 10 – O Supervised Fine-Tuning (SFT) como extensão do treinamento do modelo base em conjuntos de dados especializados. Fonte: [2].

catastrófico (*catastrophic forgetting*), onde a otimização para novas informações degrada a capacidade de processar dados anteriores [5]. Adicionalmente, objetivos abstratos — como a satisfação do usuário ou a segurança do conteúdo — são difíceis de alcançar exclusivamente com SFT. Para mitigar tais limitações, o RLHF (que compreende as duas últimas etapas do processo) introduz um modelo de recompensa que simula o julgamento humano, equilibrando a otimização da nova política com uma penalidade de similaridade em relação ao modelo original. Esse mecanismo de "objetivo duplo" assegura a evolução para atender às diretrizes sem degenerar em saídas incoerentes ou perder a naturalidade linguística conquistada no pré-treinamento [2].

Expande-se o escopo do ajuste fino através do *Instruction Tuning*, abordagem que ajusta o modelo para uma diversidade de tarefas, aprimorando sua resposta em cenários *zero-shot* [42]. Essa metodologia utiliza uma mistura massiva de tarefas (tradução, raciocínio, análise de sentimentos) verbalizadas via *templates* de linguagem natural. O objetivo

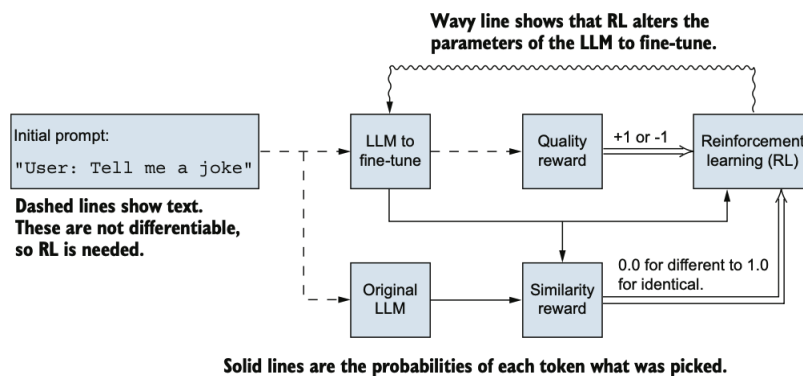


Figura 11 – RLHF: o aprendizado por reforço ajusta os parâmetros combinando pontuações de qualidade e similaridade para otimizar as saídas. Fonte: [2].

é ensinar o modelo a generalizar o conceito de "seguir instruções" para tarefas inéditas, superando a limitação de apenas completar texto. Isso marca uma transição do aprendizado guiado por exemplos (*example-driven*) para o guiado por instruções (*instruction-driven*), onde as instruções atuam como supervisão indireta, permitindo ao modelo interpretar a intenção da tarefa sem depender de exemplos rotulados específicos [43].

A eficácia dessa abordagem generalista depende criticamente da escala do modelo. Observa-se que, enquanto o ajuste de instruções potencializa a generalização em LLMs com mais de 68 bilhões de parâmetros, ele pode, paradoxalmente, degradar o desempenho de modelos menores em tarefas desconhecidas [42]. Essa disparidade sugere que, se os grandes modelos operam como generalistas robustos, os SLMs beneficiam-se predominantemente de uma estratégia de "especialistas", na qual o ajuste é direcionado a domínios de interesse específicos [42].

A literatura indica, todavia, que o ajuste de instruções é vital para "desbloquear" capacidades latentes em arquiteturas menores, permitindo que estas superem versões significativamente maiores não refinadas [34]. Evidências desse fenômeno surgem em modelos a partir de 77 milhões de parâmetros, com destaque para o Flan-T5 (11B), que superou o GPT-3 (175B) em 20 de 25 *datasets* e ultrapassou o PaLM (62B) ao integrar dados de raciocínio [42, 38]. Outro exemplo emblemático é o Med-PaLM, que atingiu performance comparável à de especialistas humanos ao refinar o modelo generalista com dados médicos [34]. Assim, modelos menores ajustados para tarefas delimitadas — como *tool calling* — conseguem igualar a eficácia de modelos massivos, oferecendo vantagens em latência e custo operacional [41].

Visando a viabilidade computacional, desenvolveram-se técnicas de ajuste fino com eficiência paramétrica (*Parameter-Efficient Fine-Tuning* - PEFT). Dentre estas, o *LoRA* (*Low-Rank Adaptation*) destaca-se por permitir a adaptação de modelos massivos sem a atualização de todos os seus parâmetros. A premissa baseia-se na hipótese de que a mudança nos pesos durante o ajuste possui uma "dimensão intrínseca" baixa [6]. Na prática,

em vez de atualizar a matriz de pesos original $W \in \mathbb{R}^{d \times k}$, o LoRA congela W e injeta duas matrizes de decomposição de baixa ordem, A e B . Durante o treinamento, apenas essas matrizes menores são atualizadas, conforme a Equação 3.1:

$$h = W_0x + \Delta Wx = W_0x + BAx \quad (3.1)$$

Onde $B \in \mathbb{R}^{d \times r}$, $A \in \mathbb{R}^{r \times k}$ e o *rank* $r \ll \min(d, k)$ [6]. Essa abordagem reduz o número de parâmetros treináveis em até 10.000 vezes e a exigência de memória de GPU em 3 vezes para modelos como o GPT-3, sem introduzir latência adicional na inferência, visto que as matrizes podem ser fundidas aos pesos originais posteriormente [6].

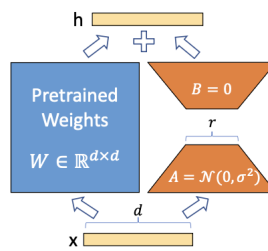


Figura 12 – Representação da técnica LoRA. Fonte: [6].

O *QLoRA* (*Quantized LoRA*) evolui esse conceito para reduzir ainda mais o uso de memória, viabilizando o ajuste de um modelo de 65B parâmetros em uma única GPU de 48GB [7]. O método opera através da retropropagação de gradientes por um modelo pré-treinado quantizado em 4 bits para os adaptadores LoRA, sustentado por três mecanismos centrais [7]:

- **4-bit NormalFloat (NF4):** Preserva a precisão dos pesos melhor do que a quantização convencional.
- **Double Quantization:** Reduz o consumo de memória ao quantizar as próprias constantes de quantização.
- **Paged Optimizers:** Gerencia picos de memória durante o processamento para evitar erros de execução.

O sucesso do QLoRA é evidenciado pela família Guanaco, que atingiu 99,3% do desempenho do ChatGPT no *benchmark* Vicuna, exigindo apenas 24 horas de ajuste fino em uma única GPU profissional [7].

3.4 Sistemas de Agentes de Inteligência Artificial

Um agente de IA define-se como um sistema computacional capaz de perceber seu ambiente, raciocinar e executar ações para alcançar objetivos [44]. Para concretizar

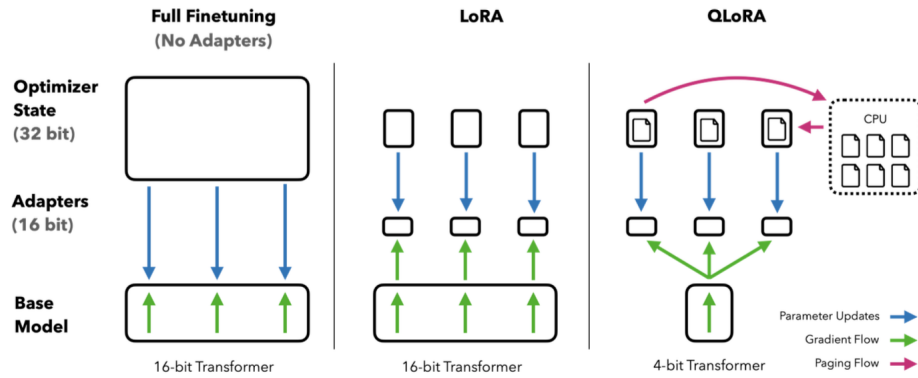


Figura 13 – Comparação de métodos: QLoRA aprimora o LoRA via quantização de 4 bits e otimizadores paginados. Fonte: [7].

tais propósitos, essa entidade assume o papel de um agente racional: aquele que, para cada sequência de percepções, seleciona a ação que maximiza sua medida de desempenho esperada, ponderando as evidências perceptivas e o conhecimento prévio acumulado [44]. O conceito estrutura-se em quatro componentes fundamentais:

- *Medida de Desempenho (Performance Measure)*: critério objetivo para avaliar o sucesso do comportamento do agente. Constitui a métrica que qualifica a sequência de estados no ambiente, gerada pelas ações do agente, como desejável ou indesejável [44].
- *Ambiente (Environment)*: contexto externo de operação. A natureza deste meio (total ou parcialmente observável, determinístico ou estocástico, estático ou dinâmico) determina a complexidade do design do sistema [44].
- *Atuadores (Actuators)*: mecanismos pelos quais o agente influencia ou altera o ambiente. Representam os meios de execução das ações selecionadas pelo programa do agente [44].
- *Sensores (Sensors)*: dispositivos ou métodos que permitem a recepção de informações sobre o estado atual do ambiente e o estado interno do sistema [44].

O programa do agente rege a relação entre esses componentes, implementando a função que mapeia o histórico de percepções em ações. Tradicionalmente baseados em regras simbólicas ou lógica formal [44], o paradigma recente direcionou o foco para agentes fundamentados em LLMs [45, 46]. Inicialmente, explorou-se a capacidade de raciocínio desses modelos via *Chain-of-Thought* (CoT), induzindo a verbalização de passos intermediários para resolução de problemas complexos, o que sugeriu o uso de LLMs como o "cérebro" do agente. Contudo, o CoT, ao depender exclusivamente de representações internas, opera como uma "caixa estática", suscetível a alucinações e propagação de erros por falta de ancoragem no mundo externo [47].

Nesse cenário, o paradigma ReAct (*Reason + Act*) propõe uma evolução arquitetural significativa. O método transcende as limitações do CoT ao sinergizar o raciocínio verbal com a tomada de decisão sequencial. Enquanto o CoT provê o planejamento e decomposição de metas (componente *Reason*), o ReAct integra a interação com o ambiente (componente *Act*) para validação. Sob uma perspectiva formal, o ReAct expande o espaço de ação tradicional: dado um sistema onde, no passo t , o agente recebe uma observação $o_t \in \mathcal{O}$ e executa uma ação $a_t \in \mathcal{A}$ seguindo uma política $\pi(a_t|c_t)$, o espaço de ação é aumentado para $\hat{\mathcal{A}} = \mathcal{A} \cup \mathcal{L}$, onde $\mathcal{L}$ denota o espaço da linguagem [46].

Nesse espaço aumentado, define-se uma ação $\hat{a}_t \in \mathcal{L}$ como um *pensamento* ou *traço de raciocínio*. Diferentemente das ações físicas ($\mathcal{A}$), o pensamento não afeta o ambiente externo nem gera *feedback* imediato. Sua função consiste em compor informações úteis raciocinando sobre o contexto atual c_t , atualizando-o para $c_{t+1} = (c_t, \hat{a}_t)$ de modo a influenciar as decisões subsequentes [46].

Essa arquitetura viabiliza um funcionamento dinâmico cíclico em duas modalidades:

1. *Raciocinar para Agir (Reason to Act)*: emprego de traços de raciocínio para criar, manter e ajustar planos de alto nível dinamicamente, tratando exceções antes da execução física [46].
2. *Agir para Raciocinar (Act to Reason)*: interação com interfaces externas (APIs, ambientes virtuais) para coletar informações adicionais que fundamentem o raciocínio em fatos observados [46].

Essa alternância contrasta com métodos de planejamento estático ou puramente reativos. Enquanto modelos *Act-Only* limitam-se no rastreamento de estados complexos e modelos *Reason-Only* (como o CoT) permanecem propensos a alucinações, a intercalação entre a geração de pensamentos ($\hat{a}_t \in \mathcal{L}$) e ações no ambiente ($a_t \in \mathcal{A}$) resulta em trajetórias de solução viáveis e factualmente ancoradas [46].

A execução robusta da etapa de "Agir" ($a_t \in \mathcal{A}$) demanda a mitigação das limitações intrínsecas aos modelos probabilísticos, notadamente a falta de acesso a dados dinâmicos e a falibilidade em cálculos exatos. A arquitetura MRKL (*Modular Reasoning, Knowledge and Language*) formaliza uma abordagem neuro-simbólica que endereça essas deficiências ao desacoplar o processamento linguístico da execução de tarefas [48].

Neste modelo, o sistema opera via uma composição de módulos governada por um *Roteador* neural e um conjunto de *Especialistas* [48]. O papel do modelo de linguagem desloca-se da geração de respostas finais para o controle de fluxo, redefinindo a arquitetura em três pontos centrais:

- *Seleção de Módulos*: o Roteador analisa a entrada em linguagem natural para identificar a intenção e selecionar o especialista adequado (modelo neural ou motor simbólico, como calculadoras ou APIs) [48].
- *Extração de Parâmetros*: o modelo extrai do texto argumentos discretos necessários para a chamada da função, atuando como interface semântica para sistemas determinísticos [48].
- *Extensibilidade Robusta*: o espaço de ação expande-se para incluir operações externas, permitindo adicionar capacidades sem retreino do modelo base, transpondo o abismo entre processamento probabilístico e raciocínio exato [48].

Enquanto a arquitetura MRKL formaliza a separação entre raciocínio neural e execução simbólica via roteadores, desenvolvimentos recentes buscam integrar o *tool calling* diretamente nos pesos do modelo, tratando o uso de ferramentas como extensão nativa da modelagem de linguagem [49, 8].

Conforme ilustrado na Figura 14, a definição de *tool calling* é formalizada através de uma interface *text-to-text* agnóstica à arquitetura [8]. O fluxo de execução converte-se em uma sequência contínua de *tokens*: dado um **input** de tarefa, o modelo gera um **tool input** seguido por um delimitador; a ferramenta processa a entrada e anexa o **tool result** à sequência; finalmente, o modelo atende a esse histórico aumentado para gerar o **output** final [8].

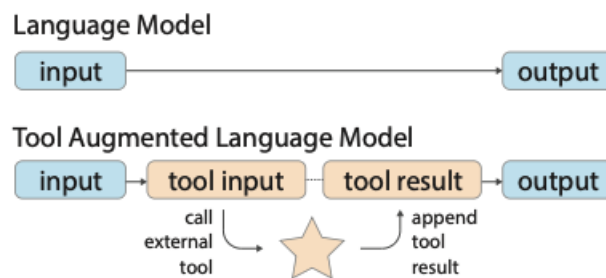


Figura 14 – Arquitetura TALM (*Tool-Augmented Language Models*): ciclo de interação entre modelo e ferramentas via interface de texto. Fonte: [8].

O experimento *Toolformer* abordou a autonomia de LLMs no uso de ferramentas (calculadoras, tradutores, APIs) [49]. A inovação central residiu na inserção de chamadas de API diretamente no texto de treino. Por exemplo, a sentença "O Joe Biden nasceu em Scranton" transforma-se em "O Joe Biden nasceu em [Search(Joe Biden birth place)] Scranton". Este trabalho fundamenta o atual "Function Calling" em modelos comerciais, evidenciando como a preparação do *dataset* permite que o modelo aprenda a orquestrar sistemas externos [49].

Nesse modelo, a chamada de ferramenta é representada linearmente como $c = (a_c, i_c)$, onde a_c é o nome da API e i_c a entrada, encapsulada por *tokens* especiais. Tal competência pode ser aprendida via SFT e ajuste fino, eliminando a necessidade de anotação humana massiva mediante um ciclo auto-supervisionado: o modelo amostra e executa chamadas, filtrando e retreinando apenas naquelas que reduzem a perplexidade da predição subsequente [8]. Essa convergência de métodos concretiza a visão neuro-simbólica ponta-a-ponta, materializando agentes de IA que utilizam ferramentas para atingir objetivos a partir da observação do ambiente [8, 49, 45].

Sistemas inteligentes complexos emergem da combinação de agentes de IA cujas interações visam um objetivo comum [50]. A comunicação pode ser colaborativa — onde agentes cooperam para metas compartilhadas inatingíveis individualmente — ou competitiva, onde um agente maximiza sua performance em detrimento de outros [51, 44].

Para abordar problemas complexos e simular cenários realistas, surgiram arquiteturas multiagentes baseadas em LLMs [52]. A adoção dessa arquitetura justifica-se pelo princípio da divisão do trabalho: tarefas complexas são decompostas em sub-rotinas atribuídas a agentes especializados [9]. Essa especialização mitiga problemas inerentes aos LLMs isolados, como alucinações e janela de contexto limitada, permitindo foco em domínios específicos. A interação pode seguir formatos estruturados, sequenciais, hierárquicos ou colaborativos (Figura 15), mimetizando equipes humanas [9].

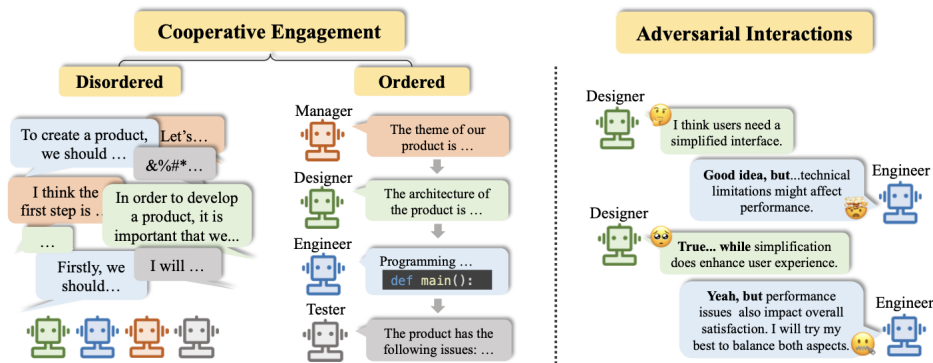


Figura 15 – Cenários de Interação para Sistemas Multiagentes com LLMs. Fonte: [9].

A aplicação da divisão do trabalho reduz, ainda, a carga cognitiva exigida de um único modelo. Ao restringir o escopo de atuação, diminui-se a probabilidade de interferências de conhecimentos não relacionados [9]. Essa compartimentalização favorece SLMs que, embora possuam menos parâmetros, podem apresentar desempenho superior em tarefas verticais quando dotados de conhecimento de domínio [41]. Tal desempenho observa-se em desenvolvimento de software, onde a simulação de papéis (gerentes, programadores, testadores) resulta em códigos mais robustos comparados àqueles gerados por agentes únicos [52].

A operacionalização dessa colaboração autônoma ocorre em frameworks de interpretação de papéis, como o CAMEL. Nessa arquitetura, coopera-se entre um agente usuário (instruções de domínio) e um agente assistente (execução técnica), mediados por um especificador de tarefas. A avaliação quantitativa evidenciou que essa abordagem de diálogo corrige autonomamente desvios e alucinações, obtendo preferência em 76% das avaliações humanas em tarefas de programação frente a agentes isolados [51].

Avançando na complexidade, o framework AutoGen possibilitou interações focadas tanto em colaboração quanto em orquestração [10]. O OptiGuide, uma de suas implementações, designa um agente "Comandante" para coordenar um "Escritor" (geração de código) e uma "Salvaguarda" (segurança) (Figura 16). Diferente de cadeias lineares, o orquestrador mantém o contexto e decide dinamicamente quando invocar validação ou correções, criando um ciclo de *feedback* autônomo que reduz a intervenção humana e aumenta a precisão do código [10].

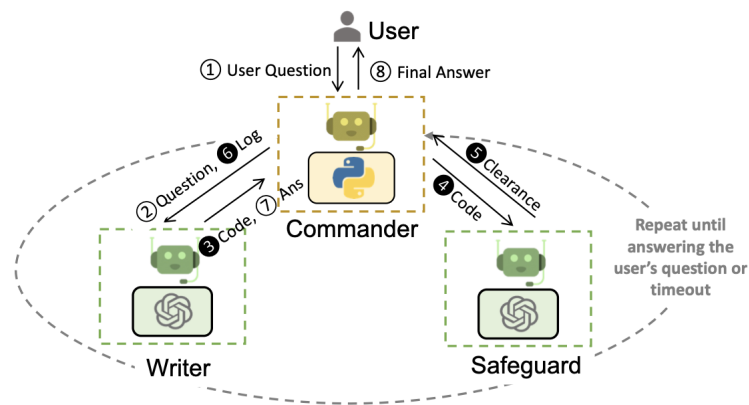


Figura 16 – Arquitetura multiagente no sistema OptiGuide: interação entre agentes de coordenação, escrita e salvaguarda. Fonte: [10].

Em uma perspectiva de maior estruturação processual, o framework MetaGPT introduziu Procedimentos Operacionais Padrão (SOPs) em sistemas multiagentes, simulando fluxos corporativos [11]. Diferentemente de abordagens de chat livre, essa arquitetura impõe comunicação via saídas estruturadas (documentos de requisitos, diagramas). Ao atribuir papéis rígidos (Gerente de Produto, Arquiteto, Engenheiro, QA), o sistema mitiga a degradação da informação comum em diálogos longos.

A eficácia dessa metodologia reside no mecanismo de *feedback* executável em tempo de execução, onde o agente de engenharia utiliza erros de compilação para ajustar o código iterativamente. Resultados nos benchmarks HumanEval e MBPP indicam taxas de sucesso superiores a 85% na geração de código funcional, superando frameworks conversacionais [11]. A Figura 17 ilustra a decomposição do fluxo em etapas sequenciais com entregáveis definidos.

Essa decomposição de tarefas estende-se a interfaces *Text-to-SQL*, onde a comple-

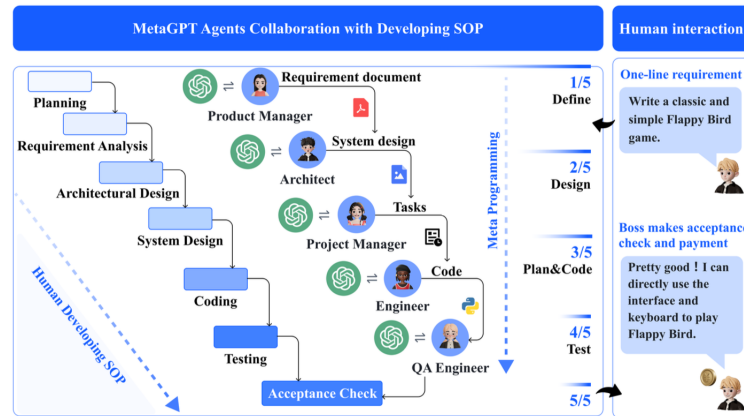


Figura 17 – Fluxo de trabalho baseado em SOPs no MetaGPT. Fonte: [11].

xidade envolve a compreensão de esquemas extensos. Arquiteturas colaborativas como o MAC-SQL estruturam o processo em três papéis: seleção, decomposição e refinamento [17]. Um agente "Seletor" filtra o conhecimento de domínio, descartando tabelas irrelevantes para manter o foco na janela de contexto; um "Decompositor" fragmenta perguntas complexas; e um "Refinador" valida e corrige o SQL gerado via ferramentas externas [17]. Testes com modelos de até 7B parâmetros ajustados demonstraram resultados comparáveis aos de LLMs generalistas, provando que a orquestração inteligente viabiliza o uso de SLMs em tarefas de alta complexidade semântica [17].

Ainda no contexto de especialização para SLMs, o framework MATS operacionaliza a verificação e correção iterativa baseada no retorno do banco de dados, dividindo a inferência em cinco papéis [53]. O diferencial reside na introdução de agentes validadores especializados — focados em seleção de colunas e condições lógicas — que comparam a intenção da pergunta com a execução preliminar. Caso inconsistências sejam detectadas, aciona-se um agente de "Correção". O treinamento por reforço com *feedback* de execução (RLE) permite que o sistema evolua e supere o desempenho de agentes isolados, mesmo utilizando modelos menores [53].

Apesar dos benefícios, sistemas multiagentes enfrentam desafios técnicos distintos, sendo a propagação de erros um dos mais críticos [50]. Diferente da alucinação isolada em agente único, em arquiteturas colaborativas o fenômeno pode gerar um efeito cascata, onde a informação incorreta é assumida como verdadeira pelos agentes subsequentes. Isso evidencia a dependência do "elo mais fraco", desafio exacerbado ao utilizarem-se modelos menores e mais propensos a erros [52].

A adoção de SLMs em arquiteturas multiagentes impõe, portanto, limites cognitivos específicos [41]. Uma barreira significativa reside na hipótese de que apenas modelos de larga escala abstraem informações semânticas complexas de domínios variados. Pragmaticamente, essa restrição manifesta-se no planejamento: embora proficientes em tarefas procedurais e código rotineiro, SLMs enfrentam dificuldades substanciais em raciocínio

arquitetural, planejamento adaptativo e resolução de erros não estruturados quando comparados a modelos generalistas [41].

3.5 Interfaces Text-to-SQL

As tarefas *Text-to-SQL* constituem um subdomínio do campo de PLN e consistem na interpretação da linguagem natural para identificação de entidades e intenções, traduzindo-as para uma consulta estruturada executável e compatível com o esquema de um banco de dados em SQL [54]. Historicamente, duas abordagens principais prevaleceram: sistemas baseados em regras e aqueles baseados em aprendizado de máquina ou profundo (*Deep Learning*). Sistemas baseados em regras dependem de índices semânticos, ontologias e grafos de conhecimento, utilizando gramáticas para gerar as consultas. Já as abordagens baseadas em aprendizado profundo codificam a entrada do usuário para treinar modelos que geram a consulta SQL. Recentemente, a convergência para arquiteturas híbridas busca mitigar as limitações individuais de cada método [54].

Os desafios na consulta em linguagem natural dividem-se em três tarefas fundamentais: (1) identificar as entidades no texto do usuário; (2) conectar tais entidades à fonte de dados para interpretar intenções; e (3) gerar a consulta estruturada final [54]. A análise semântica e a marcação de entidades focam nos termos relevantes, mas a etapa seguinte é a mais crítica, pois exige o mapeamento desses elementos para o esquema do banco de dados a fim de decifrar a intenção do usuário sob ambiguidades complexas. Por fim, a tradução concretiza a interpretação em linguagem de consulta [54]. A democratização do acesso aos dados motiva essas interfaces, oferecendo um mecanismo para que usuários não técnicos investiguem conjuntos de dados sem dominar linguagens complexas [12]. Na indústria, essa acessibilidade reduz a dependência de equipes técnicas e a latência na tomada de decisão, transformando o gestor em um "consumidor aumentado"[54].

A evolução dos LLMs alterou o paradigma de desenvolvimento dessas interfaces para arquiteturas híbridas, transitando de modelos heterogêneos e específicos para uma base unificada em *Transformers* (Figura 18).

Atualmente, a aplicação de LLMs em *Text-to-SQL* divide-se em duas vertentes: engenharia de *prompt* — com técnicas como *Retrieval-Augmented Generation* (RAG), *Chain-of-Thought*, *Few-Shot Learning* e *Tool Calling* [39, 47, 8] — e *fine-tuning*, focado na especialização do modelo e eficiência de parâmetros [5, 12]. Essa mudança arquitetural resultou em um salto qualitativo na acurácia em *benchmarks* consolidados, estabelecendo o novo estado da arte [12].

O impacto disruptivo dessas tecnologias permite dividir os *datasets* de treinamento e avaliação em "pré-LLMs" e "pós-LLMs"[12]. *Benchmarks* anteriores, como WikiSQL e, notadamente, o Spider 55, focavam na análise semântica em domínios cruzados e na corres-

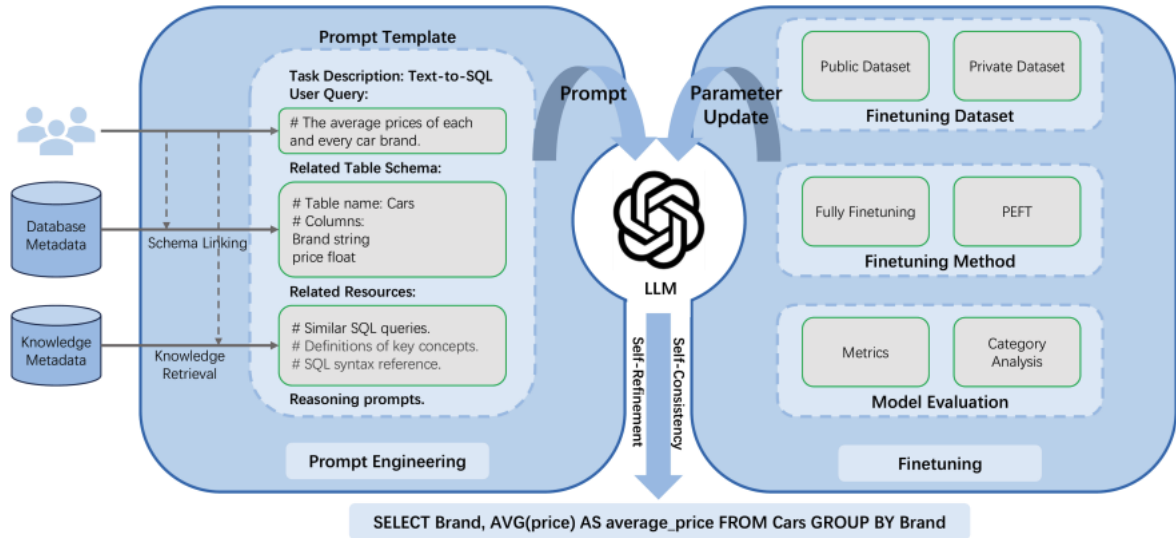


Figura 18 – Framework de LLMs em Text-to-SQL. (Fonte: 12)

pondência estrutural exata. Com a ascensão dos LLMs, surgiram métricas para mensurar capacidades emergentes e desafios corporativos reais, como o uso de conhecimento de domínio, manipulação de esquemas massivos e robustez contra ruídos, destacando-se o BIRD e o CoSQL. Contudo, o Spider permanece como a principal escolha para avaliar performance de tarefas *Text-to-SQL* [12].

Tradicionalmente, a avaliação baseia-se na Correspondência Exata (*Exact Set Match Accuracy* - EM) e na Acurácia de Execução (*Execution Accuracy* - EX) [55, 12]. A EM verifica se a consulta gerada é idêntica à de referência em componentes estruturais, sem considerar a execução [13]. No entanto, essa métrica tende a subestimar a capacidade do modelo com falsos negativos, pois penaliza variações sintáticas válidas, como a equivalência semântica entre a função **MAX** e uma ordenação seguida de **LIMIT 1** (Figura 19) [13].

```
SELECT MAX(weight) FROM dogs;
```

```
SELECT weight FROM dogs ORDER BY weight DESC LIMIT 1;
```

Figura 19 – Exemplo de falso negativo na métrica EM. As consultas buscam o peso máximo de formas distintas, levando a EM a classificar a geração como incorreta. (Fonte: 13)

Por outro lado, a EX avalia a correção funcional ao comparar os resultados da execução [55]. Embora pragmática, ela é suscetível a superestimar o desempenho (falsos positivos) quando consultas logicamente distintas retornam o mesmo resultado devido ao

estado momentâneo do banco de dados (Figura 20) [12]. Em modelos não submetidos a ajuste fino, a taxa de falsos positivos da EX pode chegar a 23,0% [13].

```
SELECT name FROM dogs;
```

```
SELECT name FROM dogs WHERE age < 10;
```

Figura 20 – Exemplo de falso positivo na métrica EX. O filtro extra altera a semântica, mas retorna o mesmo resultado vazio, gerando uma classificação incorreta de acerto. (Fonte: 13)

Para mitigar tais limitações, a literatura recente propõe a *Enhanced Tree Matching* (ETM), que avalia a equivalência semântica comparando a Árvore Sintática Abstrata (AST) das consultas [13]. O método aplica regras de normalização para formas canônicas e valida restrições de esquema (chaves primárias e unicidade), reduzindo as taxas de erro de avaliação para níveis inferiores a 3% [13].

Diante do novo paradigma de performance, as abordagens abandonam a geração monolítica em favor de estruturas decompostas e especializadas. Estratégias focadas na engenharia de fluxo de raciocínio, como o DIN-SQL [14] e o DAIL-SQL [56], fundamentam-se na premissa de que a capacidade de raciocínio dos LLMs é ampliada ao fragmentar o problema em sub-tarefas menores [14]. O DIN-SQL (*Decomposed In-Context Learning*) estrutura o processo em quatro módulos: vinculação de esquema (*schema linking*); classificação e decomposição da consulta; geração baseada na complexidade; e autocorreção (*self-correction*) [14]. Essa arquitetura demonstra que a decomposição permite que modelos generalistas superem modelos com *fine-tuning* extensivo em diversos *benchmarks* [14].

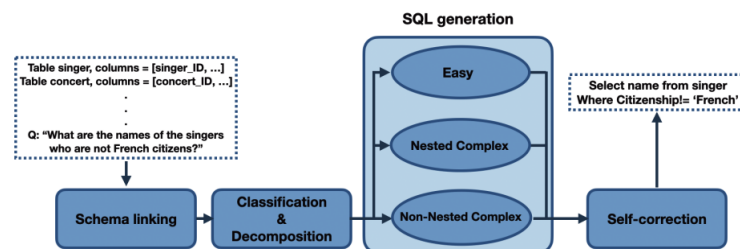


Figura 21 – Arquitetura do DIN-SQL. (Fonte: 14)

O DAIL-SQL (*Data Augmentation and Interaction Learning for SQL*) prioriza a eficiência de *tokens* e a seleção criteriosa de exemplos, assumindo que a qualidade dos casos no cenário *few-shot* e a representação do esquema são determinantes [56]. O método introduz o *DAIL Selection*, que considera as similaridades semântica e estrutural das

consultas. Além disso, valida-se a superioridade do uso de *Code Representation Prompts* (CR) via comandos DDL (`CREATE TABLE`), aproveitando o pré-treinamento dos modelos em código [56].

Entretanto, a aplicação dessas técnicas de *prompting* em SLMs apresenta restrições. Modelos da família LLaMA e Vicuna, em seu estado base, possuem capacidade de raciocínio inferior aos modelos proprietários. Embora o *Supervised Fine-Tuning* (SFT) eleve a performance desses SLMs, observa-se o efeito colateral de degradação da capacidade de *In-Context Learning*: após o ajuste, os modelos tendem a perder a habilidade de generalizar a partir de exemplos fornecidos no contexto, evidenciando que a adaptação de SLMs exige mais do que apenas treinamento em *datasets* genéricos [56].

Nesse cenário, o framework CodeS especializa o uso de SLMs para mitigar as restrições de janela de contexto. A arquitetura integra módulos de pré-processamento para otimizar a entrada: um Filtro de Esquema e um Recuperador de Valores (*Value Retriever*) via índices BM25 [15].

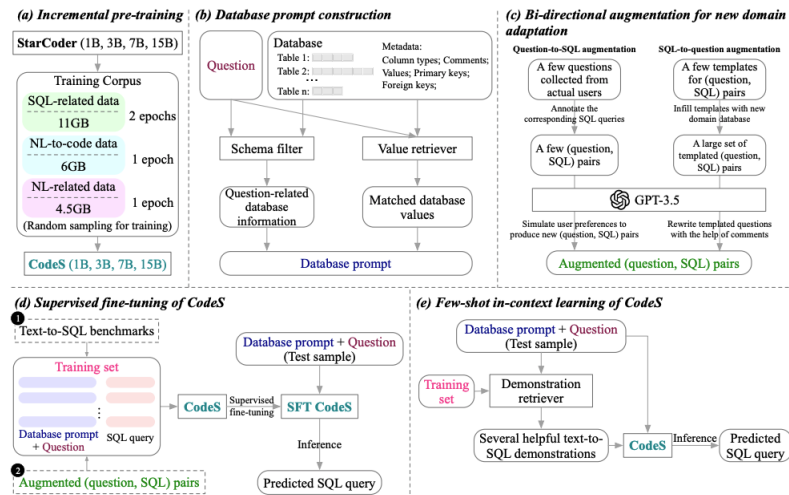


Figura 22 – Arquitetura do CodeS. (Fonte: 15)

Essa redução de dimensionalidade, combinada ao pré-treinamento incremental em SQL, permite que modelos com 3B ou 7B de parâmetros superem LLMs massivos em robustez e execução [15]. Corroborando essa viabilidade, o framework DB-GPT-Hub [16] demonstra que técnicas de eficiência de parâmetros (PEFT), como LoRA e QLoRA, permitem que SLMs alcancem desempenho comparável a modelos base significativamente maiores operando apenas via *prompting* [16]. Todavia, os ganhos do ajuste fino possuem correlação negativa com a complexidade da consulta: embora eficazes em tarefas simples e médias, a margem de ganho estreita-se em cenários complexos (Figura 23) [16]. Isso sugere que a especialização alinha o modelo à sintaxe, mas é insuficiente para prover a lógica de raciocínio necessária para aninhamentos e junções múltiplas.

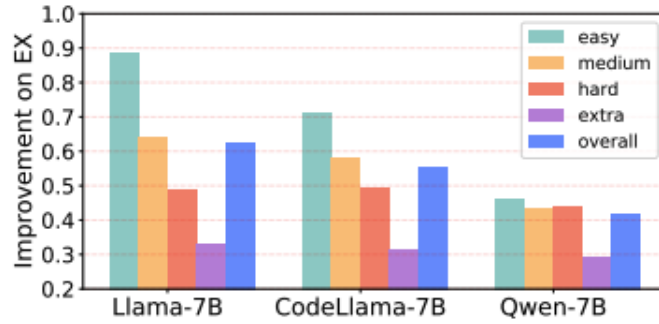


Figura 23 – Melhoria de desempenho (EX) obtida com LoRA. O ganho é menor em tarefas de alta complexidade. (Fonte: 16)

Para superar esse gargalo, a literatura recente propõe Sistemas Multiagentes (SMA), como o MAC-SQL [17], onde a responsabilidade da geração é distribuída entre unidades funcionais: um *Selector*, que mitiga ruído no esquema; um *Decomposer*, que fragmenta a pergunta via *Chain-of-Thought*; e um *Refiner*, que corrige erros com base no retorno do banco de dados [17].

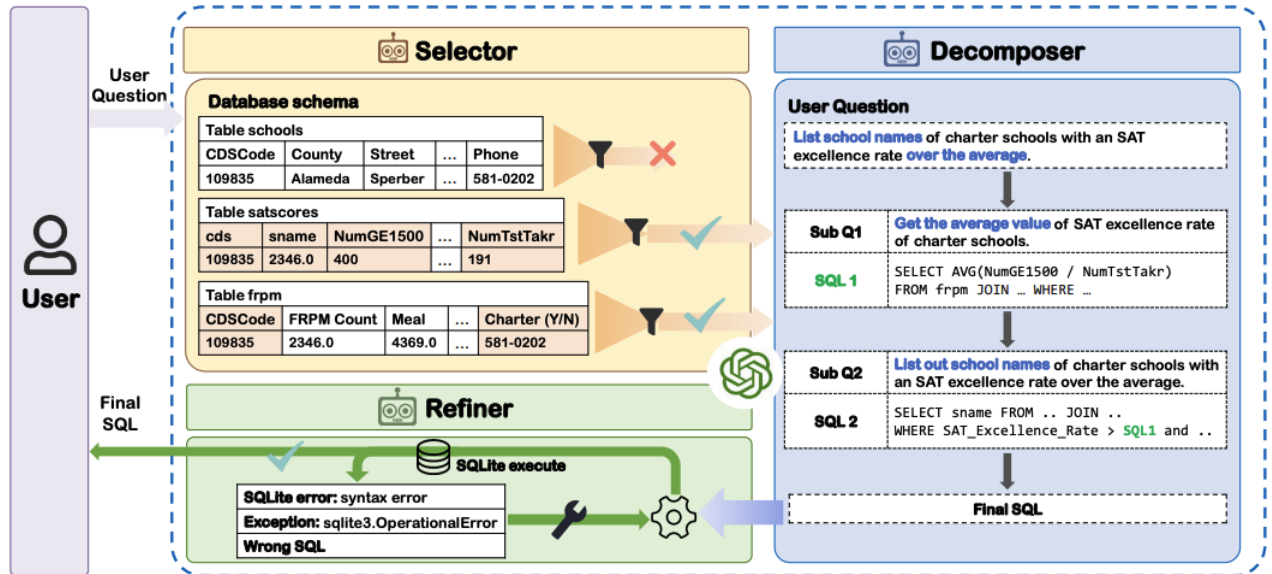


Figura 24 – Visão geral do framework MAC-SQL. (Fonte: 17)

A viabilidade dessa abordagem para modelos compactos é confirmada pelo desenvolvimento do *SQL-Llama*. Ao realizar o *fine-tuning* de um modelo de apenas 7B parâmetros com instruções específicas para o comportamento de agentes, o sistema atinge acurácia comparável à do GPT-4 em sua configuração padrão [17]. Assim, a orquestração multiagente atua como um amplificador cognitivo, permitindo que SLMs superem limitações intrínsecas e alcancem o estado da arte [17].

4 LOREM IPSUM DOLOR SIT AMET

4.1 Aliquam vestibulum fringilla lorem

a

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

4.1.1 Aliquam vestibulum fringilla lorem

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

4.1.1.1 Aliquam vestibulum fringilla lorem

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5 LECTUS LOBORTIS CONDIMENTUM

5.1 Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae

Etiam pede massa, dapibus vitae, rhoncus in, placerat posuere, odio. Vestibulum luctus commodo lacus. Morbi lacus dui, tempor sed, euismod eget, condimentum at, tortor. Phasellus aliquet odio ac lacus tempor faucibus. Praesent sed sem. Praesent iaculis. Cras rhoncus tellus sed justo ullamcorper sagittis. Donec quis orci. Sed ut tortor quis tellus euismod tincidunt. Suspendisse congue nisl eu elit. Aliquam tortor diam, tempus id, tristique eget, sodales vel, nulla. Praesent tellus mi, condimentum sed, viverra at, consectetur quis, lectus. In auctor vehicula orci. Sed pede sapien, euismod in, suscipit in, pharetra placerat, metus. Vivamus commodo dui non odio. Donec et felis.

Etiam suscipit aliquam arcu. Aliquam sit amet est ac purus bibendum congue. Sed in eros. Morbi non orci. Pellentesque mattis lacinia elit. Fusce molestie velit in ligula. Nullam et orci vitae nibh vulputate auctor. Aliquam eget purus. Nulla auctor wisi sed ipsum. Morbi porttitor tellus ac enim. Fusce ornare. Proin ipsum enim, tincidunt in, ornare venenatis, molestie a, augue. Donec vel pede in lacus sagittis porta. Sed hendrerit ipsum quis nisl. Suspendisse quis massa ac nibh pretium cursus. Sed sodales. Nam eu neque quis pede dignissim ornare. Maecenas eu purus ac urna tincidunt congue.

6 NAM SED TELLUS SIT AMET LECTUS URNA ULLAM-CORPER TRISTIQUE INTERDUM ELEMENTUM

6.1 Pellentesque sit amet pede ac sem eleifend consectetuer

Maecenas non massa. Vestibulum pharetra nulla at lorem. Duis quis quam id lacus dapibus interdum. Nulla lorem. Donec ut ante quis dolor bibendum condimentum. Etiam egestas tortor vitae lacus. Praesent cursus. Mauris bibendum pede at elit. Morbi et felis a lectus interdum facilisis. Sed suscipit gravida turpis. Nulla at lectus. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Praesent nonummy luctus nibh. Proin turpis nunc, congue eu, egestas ut, fringilla at, tellus. In hac habitasse platea dictumst.

7 CONCLUSÃO

Sed consequat tellus et tortor. Ut tempor laoreet quam. Nullam id wisi a libero tristique semper. Nullam nisl massa, rutrum ut, egestas semper, mollis id, leo. Nulla ac massa eu risus blandit mattis. Mauris ut nunc. In hac habitasse platea dictumst. Aliquam eget tortor. Quisque dapibus pede in erat. Nunc enim. In dui nulla, commodo at, consectetur nec, malesuada nec, elit. Aliquam ornare tellus eu urna. Sed nec metus. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas.

Phasellus id magna. Duis malesuada interdum arcu. Integer metus. Morbi pulvinar pellentesque mi. Suspendisse sed est eu magna molestie egestas. Quisque mi lorem, pulvinar eget, egestas quis, luctus at, ante. Proin auctor vehicula purus. Fusce ac nisl aliquam ante hendrerit pellentesque. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Morbi wisi. Etiam arcu mauris, facilisis sed, eleifend non, nonummy ut, pede. Cras ut lacus tempor metus mollis placerat. Vivamus eu tortor vel metus interdum malesuada.

Sed eleifend, eros sit amet faucibus elementum, urna sapien consectetur mauris, quis egestas leo justo non risus. Morbi non felis ac libero vulputate fringilla. Mauris libero eros, lacinia non, sodales quis, dapibus porttitor, pede. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Morbi dapibus mauris condimentum nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Etiam sit amet erat. Nulla varius. Etiam tincidunt dui vitae turpis. Donec leo. Morbi vulputate convallis est. Integer aliquet. Pellentesque aliquet sodales urna.

REFERÊNCIAS

- [1] ARAUJO, L. C. *Configuração: uma perspectiva de arquitetura da informação da escola de Brasília*. Dissertação (Mestrado) — Universidade de Brasília, Brasília, Março 2012.
- [2] FARRIS, D.; RAFF, E.; BIDERMAN, S. *How Large Language Models Work*. [S.l.]: Manning, 2025.
- [3] VASWANI, A. et al. Attention is all you need. In: GUYON, I. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf>.
- [4] CASELI, H. M.; NUNES, M. G. V. (Ed.). *Processamento de Linguagem Natural: Conceitos, Técnicas e Aplicações em Português*. 2. ed. BPLN, 2024. ISBN 978-65-00-95750-1. Disponível em: <<https://brasileiraspln.com/livro-pln/2a-edicao/>>.
- [5] OUYANG, L. et al. Training language models to follow instructions with human feedback. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2022. (NIPS '22). ISBN 9781713871088.
- [6] HU, E. J. et al. Lora: Low-rank adaptation of large language models. In: *ICLR*. OpenReview.net, 2022. Disponível em: <<http://dblp.uni-trier.de/db/conf/iclr/iclr2022.html#HuSWALWWC22>>.
- [7] DETTMERS, T. et al. Qlora: efficient finetuning of quantized llms. In: *Proceedings of the 37th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2023. (NIPS '23).
- [8] PARISI, A.; ZHAO, Y.; FIEDEL, N. *TALM: Tool Augmented Language Models*. 2022. Disponível em: <<https://arxiv.org/abs/2205.12255>>.
- [9] XI, Z. et al. The rise and potential of large language model based agents: a survey. *Science China Information Sciences*, v. 68, n. 2, p. 121101, Jan 2025. ISSN 1869-1919. Disponível em: <<https://doi.org/10.1007/s11432-024-4222-0>>.
- [10] WU, Q. et al. Autogen: Enabling next-gen llm applications via multi-agent conversation. In: *COLM 2024*. [s.n.], 2024. Disponível em: <<https://www.microsoft.com/en-us/research/publication/autogen-enabling-next-gen-llm-applications-via-multi-agent-conversation-framework/>>.
- [11] HONG, S. et al. MetaGPT: Meta programming for a multi-agent collaborative framework. In: *The Twelfth International Conference on Learning Representations*. [s.n.], 2024. Disponível em: <<https://openreview.net/forum?id=VtmBAGCN7o>>.
- [12] SHI, L. et al. A survey on employing large language models for text-to-sql tasks. *ACM Comput. Surv.*, Association for Computing Machinery, New York, NY, USA, v. 58, n. 2, set. 2025. ISSN 0360-0300. Disponível em: <<https://doi.org/10.1145/3737873>>.

- [13] ASCOLI, B.; KANDIKONDA, R.; CHOI, J. D. ETM: modern insights into perspective on text-to-sql evaluation in the age of large language models. *Future Internet*, v. 17, n. 8, p. 325, 2025. Disponível em: <<https://doi.org/10.3390/fi17080325>>.
- [14] POURREZA, M.; RAFIEI, D. DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: *Thirty-seventh Conference on Neural Information Processing Systems*. [s.n.], 2023. Disponível em: <<https://openreview.net/forum?id=p53QDxSIc5>>.
- [15] LI, H. et al. Codes: Towards building open-source language models for text-to-sql. *Proc. ACM Manag. Data*, Association for Computing Machinery, New York, NY, USA, v. 2, n. 3, 2024. Disponível em: <<https://doi.org/10.1145/3654930>>.
- [16] ZHOU, F. et al. *DB-GPT-Hub: Towards Open Benchmarking Text-to-SQL Empowered by Large Language Models*. 2024. Disponível em: <<https://openreview.net/forum?id=NmILZXKcOi>>.
- [17] WANG, B. et al. Mac-sql: A multi-agent collaborative framework for text-to-sql. In: *Proceedings of the 31st International Conference on Computational Linguistics*. [S.l.: s.n.], 2025. p. 540–557.
- [18] ABNTEX2; ARAUJO, L. C. *A classe abntex2: Modelo canônico de trabalhos acadêmicos brasileiros compatível com as normas ABNT NBR 14724:2011, ABNT NBR 6024:2012 e outras*. [S.l.], 2013. Disponível em: <<http://abntex2.googlecode.com/>>.
- [19] ABNTEX2. *Como customizar o abnTeX2*. 2013. Wiki do abnTeX2. Disponível em: <<https://code.google.com/p/abntex2/wiki/ComoCustomizar>>. Acesso em: 23.3.2013.
- [20] ABNTEX2; ARAUJO, L. C. *O pacote abntex2cite: Estilos bibliográficos compatíveis com a ABNT NBR 6023*. [S.l.], 2013. Disponível em: <<http://abntex2.googlecode.com/>>.
- [21] ABNTEX2; ARAUJO, L. C. *O pacote abntex2cite: tópicos específicos da ABNT NBR 10520:2002 e o estilo bibliográfico alfabético (sistema autor-data)*. [S.l.], 2013. Disponível em: <<http://abntex2.googlecode.com/>>.
- [22] WILSON, P.; MADSEN, L. *The Memoir Class for Configurable Typesetting - User Guide*. Normandy Park, WA, 2010. Disponível em: <<http://mirrors.ctan.org/macros/latex/contrib/memoir/memman.pdf>>. Acesso em: 19.12.2012.
- [23] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 10520: Informação e documentação — apresentação de citações em documentos*. Rio de Janeiro, 2002. 7 p.
- [24] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 14724: Informação e documentação — trabalhos acadêmicos — apresentação*. Rio de Janeiro, 2011. 15 p.
- [25] van GIGCH, J. P.; PIPINO, L. L. In search for a paradigm for the discipline of information systems. *Future Computing Systems*, v. 1, n. 1, p. 71–97, 1986.

- [26] ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. *NBR 6024*: Numeração progressiva das seções de um documento. Rio de Janeiro, 2012. 4 p.
- [27] BRAAMS, J. *Babel, a multilingual package for use with LATEX's standard document classes*. [S.l.], 2008. Disponível em: <<http://mirrors.ctan.org/info/babel/babel.pdf>>. Acesso em: 17.2.2013.
- [28] AMARATUNGA, T. (Ed.). *Understanding Large Language Models*. [S.l.]: Apress, 2023.
- [29] JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3rd. ed. [s.n.], 2025. Disponível em: <<https://web.stanford.edu/~jurafsky/slp3/>>.
- [30] GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. *Deep Learning*. [S.l.]: MIT Press, 2016. <<http://www.deeplearningbook.org>>.
- [31] DEVLIN, J. et al. BERT: Pre-training of deep bidirectional transformers for language understanding. In: BURSTEIN, J.; DORAN, C.; SOLORIO, T. (Ed.). *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, 2019. p. 4171–4186. Disponível em: <<https://aclanthology.org/N19-1423/>>.
- [32] KAPLAN, J. et al. *Scaling Laws for Neural Language Models*. 2020. Disponível em: <<https://arxiv.org/abs/2001.08361>>.
- [33] WEI, J. et al. Emergent abilities of large language models. *Trans. Mach. Learn. Res.*, v. 2022, 2022. Disponível em: <<http://dblp.uni-trier.de/db/journals/tmlr/tmlr2022.html#WeiTBRZBYBZMCHVLDF22>>.
- [34] ZHAO, W. X. et al. *A Survey of Large Language Models*. 2025. Disponível em: <<https://arxiv.org/abs/2303.18223>>.
- [35] HOFFMANN, J. et al. Training compute-optimal large language models. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2022. (NIPS '22). ISBN 9781713871088.
- [36] TOUVRON, H. et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. Disponível em: <<https://arxiv.org/abs/2302.13971>>.
- [37] GUNASEKAR, S. et al. *Textbooks Are All You Need*. 2023. Disponível em: <<https://arxiv.org/abs/2306.11644>>.
- [38] CHUNG, H. W. et al. Scaling instruction-finetuned language models. *J. Mach. Learn. Res.*, JMLR.org, v. 25, n. 1, jan. 2024. ISSN 1532-4435.
- [39] WANG, F. et al. A comprehensive survey of small language models in the era of large language models: Techniques, enhancements, applications, collaboration with llms, and trustworthiness. *ACM Trans. Intell. Syst. Technol.*, Association for Computing Machinery, New York, NY, USA, v. 16, n. 6, nov. 2025. ISSN 2157-6904. Disponível em: <<https://doi.org/10.1145/3768165>>.

- [40] GARG, A. et al. *Emissions and Performance Trade-off Between Small and Large Language Models*. 2025. Disponível em: <<https://arxiv.org/abs/2601.08844>>.
- [41] BELCAK, P. et al. *Small Language Models are the Future of Agentic AI*. 2025. Disponível em: <<https://arxiv.org/abs/2506.02153>>.
- [42] WEI, J. et al. Finetuned language models are zero-shot learners. In: *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. Disponível em: <<https://openreview.net/forum?id=gEZrGCozdqR>>.
- [43] LOU, R.; ZHANG, K.; YIN, W. Large language model instruction following: A survey of progresses and challenges. *Computational Linguistics*, MIT Press, Cambridge, MA, v. 50, n. 3, p. 1053–1095, set. 2024. Disponível em: <<https://aclanthology.org/2024.cl-3.7/>>.
- [44] RUSSELL, S.; NORVIG, P. *Artificial Intelligence: A Modern Approach (4th Edition)*. Pearson, 2020. ISBN 9780134610993. Disponível em: <<http://aima.cs.berkeley.edu/>>.
- [45] KONG, Y. et al. TPTU-v2: Boosting task planning and tool usage of large language model-based agents in real-world industry systems. In: DERNONCOURT, F.; PREOTIU-PIETRO, D.; SHIMORINA, A. (Ed.). *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Miami, Florida, US: Association for Computational Linguistics, 2024. p. 371–385. Disponível em: <<https://aclanthology.org/2024.emnlp-industry.27/>>.
- [46] YAO, S. et al. React: Synergizing reasoning and acting in language models. In: *ICLR*. OpenReview.net, 2023. Disponível em: <<http://dblp.uni-trier.de/db/conf/iclr/iclr2023.html#YaoZYDSN023>>.
- [47] WEI, J. et al. Chain-of-thought prompting elicits reasoning in large language models. In: *Proceedings of the 36th International Conference on Neural Information Processing Systems*. Red Hook, NY, USA: Curran Associates Inc., 2022. (NIPS '22). ISBN 9781713871088.
- [48] KARPAS, E. et al. MRKL systems: A modular, neuro-symbolic architecture that combines large language models, external knowledge sources and discrete reasoning. *CoRR*, abs/2205.00445, 2022. Disponível em: <<https://doi.org/10.48550/arXiv.2205.00445>>.
- [49] SCHICK, T. et al. Toolformer: Language models can teach themselves to use tools. *CoRR*, abs/2302.04761, 2023. Disponível em: <<https://doi.org/10.48550/arXiv.2302.04761>>.
- [50] HULTEN, G. *Building Intelligent Systems: A Guide to Machine Learning Engineering*. 1st. ed. USA: Apress, 2018. ISBN 1484234316.
- [51] LI, G. et al. Camel: Communicative agents for "mind" exploration of large language model society. In: OH, A. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2023. v. 36, p. 51991–52008. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2023/file/a3621ee907def47c1b952ade25c67698-Paper-Conference.pdf>.

- [52] GUO, T. et al. Large language model based multi-agents: A survey of progress and challenges. In: LARSON, K. (Ed.). *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24*. International Joint Conferences on Artificial Intelligence Organization, 2024. p. 8048–8057. Survey Track. Disponível em: <<https://doi.org/10.24963/ijcai.2024/890>>.
- [53] HOANG, T. D. et al. *A Multi-agent Text2SQL Framework using Small Language Models and Execution Feedback*. 2025. Disponível em: <<https://arxiv.org/abs/2512.18622>>.
- [54] QUAMAR, A. et al. Natural language interfaces to data. *Found. Trends Databases*, Now Publishers Inc., Hanover, MA, USA, v. 11, n. 4, p. 319–414, maio 2022. ISSN 1931-7883. Disponível em: <<https://doi.org/10.1561/19000000078>>.
- [55] YU, T. et al. Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: RILOFF, E. et al. (Ed.). *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Brussels, Belgium: Association for Computational Linguistics, 2018. p. 3911–3921. Disponível em: <<https://aclanthology.org/D18-1425/>>.
- [56] GAO, D. et al. Text-to-sql empowered by large language models: A benchmark evaluation. *Proc. VLDB Endow.*, v. 17, n. 5, p. 1132–1145, 2024. Disponível em: <<https://www.vldb.org/pvldb/vol17/p1132-gao.pdf>>.

Apêndices

APÊNDICE A – QUISQUE LIBERO JUSTO

Quisque facilisis auctor sapien. Pellentesque gravida hendrerit lectus. Mauris rutrum sodales sapien. Fusce hendrerit sem vel lorem. Integer pellentesque massa vel augue. Integer elit tortor, feugiat quis, sagittis et, ornare non, lacus. Vestibulum posuere pellentesque eros. Quisque venenatis ipsum dictum nulla. Aliquam quis quam non metus eleifend interdum. Nam eget sapien ac mauris malesuada adipiscing. Etiam eleifend neque sed quam. Nulla facilisi. Proin a ligula. Sed id dui eu nibh egestas tincidunt. Suspendisse arcu.

Anexos

ANEXO A – MORBI ULTRICES RUTRUM LOREM.

Sed mattis, erat sit amet gravida malesuada, elit augue egestas diam, tempus scelerisque nunc nisl vitae libero. Sed consequat feugiat massa. Nunc porta, eros in eleifend varius, erat leo rutrum dui, non convallis lectus orci ut nibh. Sed lorem massa, nonummy quis, egestas id, condimentum at, nisl. Maecenas at nibh. Aliquam et augue at nunc pellentesque ullamcorper. Duis nisl nibh, laoreet suscipit, convallis ut, rutrum id, enim. Phasellus odio. Nulla nulla elit, molestie non, scelerisque at, vestibulum eu, nulla. Ut odio nisl, facilisis id, mollis et, scelerisque nec, enim. Aenean sem leo, pellentesque sit amet, scelerisque sit amet, vehicula pellentesque, sapien.

TRABALHOS PUBLICADOS PELO AUTOR

Trabalhos publicados pelo autor durante o programa.

Publicações principais do trabalho.

1. Jose da silva, autor2 da silva, orientador da silva, **Título do artigo**, local onde foi publicado, mês/ano, editora, número de página, isbn, etc. (Qualis CC 2017, xx)
2. Jose da silva, autor2 da silva, orientador da silva, **Título do artigo**, local onde foi publicado, mês/ano, editora, número de página, isbn, etc. (Qualis CC 2017, xx)
3. Jose da silva, autor2 da silva, orientador da silva, **Título do artigo**, local onde foi publicado, mês/ano, editora, número de página, isbn, etc. (Qualis CC 2017, xx)

Publicações complementares.

1. Jose da silva, autor2 da silva, orientador da silva, **Título do artigo**, local onde foi publicado, mês/ano, editora, número de página, isbn, etc. (Qualis CC 2017, xx)
2. Jose da silva, autor2 da silva, orientador da silva, etc. **Título do artigo**, local onde foi publicado, mês/ano, editora, número de página, isbn, (Qualis CC 2017, xx)

ÍNDICE

Adobe Illustrator, 18

Adobe Photoshop, 18

alíneas, 19

citações

 diretas, 16

 simples, 16

CorelDraw, 18

espaçamento

 do primeiro parágrafo, 20

 dos parágrafos, 20

 entre as linhas, 21

 entre os parágrafos, 20

expressões matemáticas, 19

figuras, 17

filosofia, 17

Gimp, 18

incisos, 19

InkScape, 18

sinopse de capítulo, 16

subalíneas, 19

tabelas, 17